Software and Systems Modeling (2026) 25:579–614
https://doi.org/10.1007/s10270-025-01308-y

REGULAR PAPER

A model and workflow-driven approach for engineering domain-specific low-code platforms and applications

Francisco Martínez-Lasaca[1,2] · Pablo Díez[2] · Esther Guerra[1] · Juan de Lara[1]

Received: 14 February 2025 / Revised: 23 May 2025 / Accepted: 16 June 2025 / Published online: 18 July 2025
© The Author(s) 2025

Abstract
The need to produce software quicker and in greater quantities continues to grow, while the market for professional programmers struggles to meet the rising demand. Low-code development platforms (LCDPs) have been proposed as a solution that enables individuals without formal training to develop applications. However, current LCDPs typically rely on proprietary, fixed code generation processes, which limit flexibility, hinder interoperability, and risk vendor lock-in. To address these limitations, this paper introduces Dandelion+, a cloud-based low-code platform designed to create domain-specific LCDPs and applications within them. Our approach is highly model-driven, from defining the scaffolds of low-code platforms (e.g., models, meta-models, users, and roles) to creating applications. In particular, applications are powered by PLATFLOW, a platform-aware graphical workflow language. Comprising both model operations and low-code-focused nodes, PLATFLOW offers flexible control over the code generation process that drives LCDPs. We conduct two case studies to evaluate our approach. First, we confront Dandelion+ with other low-code platforms to design interactive applications and produce artifacts. Second, we report on the benefits of Dandelion+ in the industrial context of UGROUND, a Spanish software firm that uses low code in its projects.

Keywords Low-code platforms · Model-driven engineering · Workflow languages

1 Introduction

Since the early days of computer science, researchers and practitioners have sought methods to improve productivity and quality in software development. In recent decades, this effort has intensified as society's reliance on software has grown, creating a need to produce software quicker and in greater quantities. This demand puts pressure on the programming industry, which is compelled to expand its workforce. Moreover, there are a growing number of non-professional programmers who need to carry out programming tasks as part of their work. Even if individuals are becoming increasingly tech-savvy and routinely use technology for complex tasks, many lack formal training in computer science. As a result, they cannot harness standard tools and IDEs designed for professional developers to build their own automation solutions.

Model-driven engineering (MDE) has been proposed as a proponent for enhancing productivity, as it increases the level of abstraction and enables automation [5]. Still, automation solutions built with MDE frequently target professional developers. Conversely, low-code development platforms (LCDPs) aim to cover *"the other side of the coin"* [10] by targeting citizen developers, offering them (often graphical) languages and tooling to complete small programming tasks [59]. For this reason, LCDPs focus on providing a user-friendly development experience in cloud-based settings, featuring zero-cost installation and hosting the generated applications.

Communicated by Peter Clarke.

✉ Francisco Martínez-Lasaca
francisco.martinezl01@estudiante.uam.es ;
fmartinez@uground.com

Pablo Díez
pdiez@uground.com

Esther Guerra
esther.guerra@uam.es

Juan de Lara
juan.delara@uam.es

[1] Universidad Autónoma de Madrid, Madrid, Spain

[2] UGROUND, Madrid, Spain

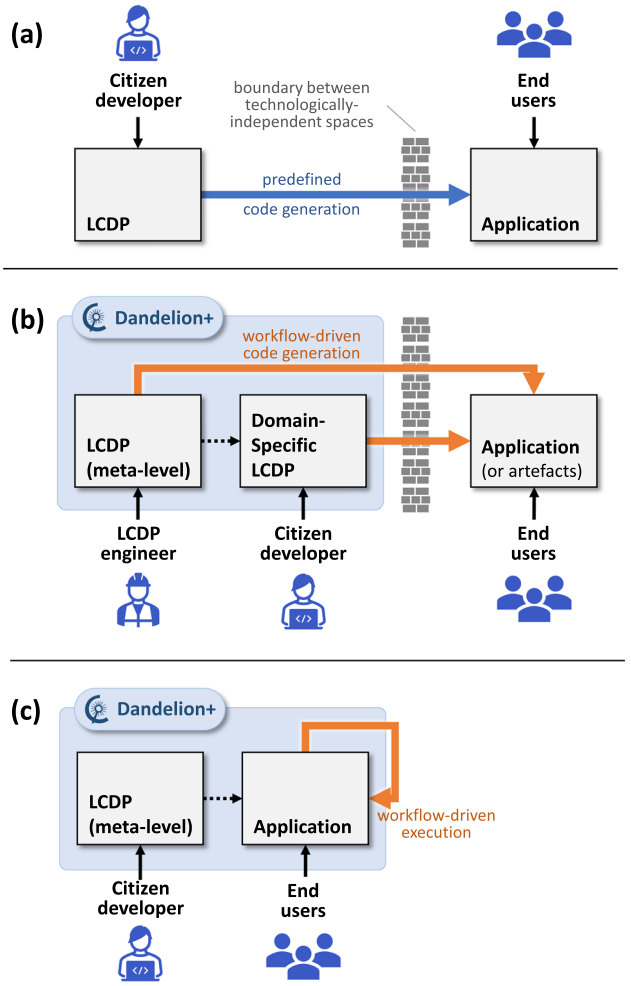


Fig. 1 Approaches for building applications within low-code environments. **a** The regular LCDP approach, using a predefined code generator. **b** Dandelion+ approach, enabling building domain-specific LCDPs through replacing the fixed generation step with domain-specific modeling and workflow execution. **c** An alternative approach, also supported by Dandelion+, that does not depend on code generation, but directly runs the final application within the platform

Many LCDPs exist today, including OutSystems [40], Mendix [34], Microsoft Power Apps [35], and Google AppSheet [20]. They follow the development process depicted in Fig. 1a. First, citizen developers use an LCDP to build the blueprints of an application. Then, a predefined code generator is executed to generate the application. Finally, the application is deployed to the cloud, becoming available to end users. However, since most platforms are proprietary and closed-source, the generation step is usually fixed and platform-specific, hindering interoperability and causing vendor lock-in.

As discussed in [10], LCDPs can benefit from adopting MDE techniques, including modeling and meta-modeling (to describe application structure), as well as model transformations and code generation (to describe application behavior). On the one hand, explicit meta-models for LCDPs would enable interoperability. On the other hand, explicit meta-modeling support could overcome the limitation of most LCDPs today, so that they could be used to build domain-specific LCDPs (e.g., low-code platforms for defining mobile apps, games, or data science workflows [48]), which then can generate apps (or artifacts) for the end users via code generation (cf. Fig. 1b). As Fig. 1c shows, code generation is not the only way to define applications. Instead, an MDE-enabled LCDP like Dandelion+ can rely on modeling and transformations to directly define final applications and run them within the same environment without generating code.

To support these scenarios involving the construction of LCDPs and applications, we argue that merely porting MDE technologies to the web is insufficient. On the one hand, specific low-code concepts should be incorporated into structural (meta-)modeling languages (e.g., roles, users, platforms, or files). On the other, we posit that model management operations within LCDPs should be orchestrated using dedicated languages that also incorporate low-code concepts, like managing entities or consuming external APIs.

To tackle these issues, we propose **Dandelion+**, a cloud, MDE-based low-code platform that can be used to build domain-specific LCDPs and final applications. Dandelion+ is founded on MDE principles, featuring an agnostic meta-modeling language that is complemented by concepts specific to low code. To describe the behavior of LCDPs or final applications, Dandelion+ offers a workflow language—called **PlatFlow**—that orchestrates both MDE tasks (e.g., model validation, model-to-model, or model-to-text transformations) and low-code-specific tasks (e.g., managing entities or calling APIs). This paper presents the rationale behind Dandelion+, along with its architecture and tooling. To validate the proposal, we present an evaluation where Dandelion+ is confronted with other low-code tools and extant industrial approaches to develop low-code platforms.

The contributions of this article are:

1. An MDE-based architecture for low-code development, which can be used to define both domain-specific LCDPs and applications.
2. PlatFlow, a workflow language designed to drive the behavior of low-code applications.
3. Dandelion+, a tool that realizes the previous ideas.
4. A validation of the approach, which confronts Dandelion+ to other low-code platforms and demonstrates the benefits of the tool in industrial processes.

The rest of the article is structured as follows. Section 2 examines related work, conducting a gap analysis. Section 3 presents a running example. Section 4 provides an overview of the approach. Then, Sect. 5 details our approach to designing the structure of low-code platforms, and Sect. 6 describes

how to infuse behavior into those platforms via our workflow language PLATFLOW. Next, Sect. 7 describes the architecture and tool support of Dandelion+. Finally, Sect. 8 evaluates our approach and Sect. 9 ends with the conclusions and future work.

2 Related work

In this section, we examine works related to low-code platforms 2.1 and workflow languages 2.2. Then, in 2.3, we confront the features and limitations of these works with those of Dandelion+ in a gap analysis.

2.1 Low-code platforms

Low-code platforms are automation solutions based on *domain-specific languages* (DSLs) [62]. Hence, we can consider *language workbenches* as the precursors of these platforms, as they enable building DSL-based solutions. Examples of such workbenches include MetaEdit+ [24], AToM^3 [12], or tools emerged around the Eclipse Modeling Framework (EMF) [54] like Sirius [53], Xtext [63], or the GEMOC Studio [19]. These tools are desktop-based, target technical developers, and incorporate modeling into their development processes—often through graphical languages.

Later, the improvement and increase of cloud services encouraged the migration of desktop-based MDE tooling to the cloud, resulting in language workbenches for the web such as WebGME [30], AToMPM [8], Pyro [67], Sirius Web [58], or EMF.cloud [56]. The main selling point of these tools is their ubiquity: being web-based, they can be accessed from any device, eliminating the need for local installation.

Concurrently, companies—especially large firms—started to invest on low-code platforms, typically exploited under a platform-as-a-service (PaaS) business model [10, 47]. These platforms provide development environments where traditional coding is replaced by approaches more amenable to non-technical users, such as drag-and-drop interfaces of premade components. Notable examples are OutSystems [40], Mendix [34], Zoho Creator [66], Microsoft Power Apps [35], Google AppSheet [20],[1] Kissflow [25], Salesforce Platform [49],[2] Appian [2], or Caspio [6]. The next paragraphs discuss the salient features of these platforms.

OutSystems [40] is one of the most prominent low-code platforms. Application development takes place in *Service Studio*, its dedicated desktop IDE. Each application is made of modules which, in turn, are divided into interface, logic, and data. The interface editor features many visual widgets, including graphs, forms, and lists, which can be arranged in a drag-and-drop graphical editor. Entities structure is managed through databases. Regarding logic, OutSystems differentiates between *client* and *server* actions. The former are used to manipulate the UI and can be triggered by UI events, while the latter may perform small backend computations.

OutSystems can expose server actions as services using SOAP, REST, or SAP. After every development iteration, the final application must be generated, compiled, and deployed to the cloud using a predefined and closed-source generation process. The result is a cross-platform application, covering web and mobile. The strength of OutSystems lies in its rich set of visual components, its numerous integrations with third parties, and its ease of use.

Mendix [34] is another key proponent of low-code development. Its IDE is called *Mendix Studio*, and it is desktop-based. In Mendix, applications (or *modules*) contain domain models, security configuration, images, pages, and (micro-)flows. Mendix follows a closer approach to modeling, as the domain model manages entities and relationships, unlike OutSystems which relies on database schemas. For behavior, Mendix uses *microflows*, which are built using a drag-and-drop editor and manage objects, lists, variables, and client activities. Mendix also supports integration with external services like SAP and allows data manipulation via REST services. As with OutSystems, applications are built before they are deployed to the cloud.

While OutSystems and Mendix focus on creating interactive webs (typically enterprise applications), **Zoho Creator** [66] covers the niche of data-intensive applications. It offers over 500 integrations with services like Salesforce, GitHub, and Stripe, which can be invoked through workflows triggered by CRON jobs or events. The platform uses the proprietary scripting language *Deluge*[3] to uniformize the integration with third-party services in the workflows. The salient feature of the platform is building dashboards. The development is agile, as dashboard construction is guided by a recommender system that exploits the definition of the application's domain. The visual representation is also highly customizable thanks to the *Zoho Markup Language* (ZML),[4] a proprietary, XML-based language for describing the visual structure of widgets within dashboards.

The proposal from Microsoft is **Power Apps** [35], a low-code platform deeply integrated within the company's ecosystem, allowing the development of applications connected with services like Power BI, Power Automate, or Microsoft Dataverse (i.e., Microsoft's centralized data storage). The platform relies on *Power Fx*, a formula-based

[1] Google App Maker closed in 2021.

[2] Previously, it was called Salesforce App Cloud.

[3] https://www.zoho.com/deluge.

[4] https://help.zoho.com/portal/en/kb/creator/developer-guide/zml-guide/articles/zml-an-overview.

language similar to Excel's, for application behavior. The language is declarative, and it allows a reactive definition of applications' behavior. A differentiating trait of Power Apps is that it features both *robotic process automation* (RPA) and *digital process automation* (DPA): RPA automates repetitive tasks by replicating user actions, while DPA supports a traditional approach of workflows for backend development. Following the same approach as Zoho Creator with ZML, the structure of the visual widgets can be customized using *Adaptive Cards*,[5] a JSON-based language with templating facilities.

While many low-code platforms aim to produce highly customizable solutions, the scope of **Google AppSheet** [20] is different: bringing user spreadsheets (e.g., Google Sheets) alive as dashboards. Given a spreadsheet, the platform derives a data schema from the data contained in the spreadsheet. This way, every worksheet maps to the definition of an entity; columns within the worksheet define fields of the entity; and every row becomes an instance. The look and feel of the entities can be customized through a collection of predefined widgets, including calendars, tables, galleries, and forms. Moreover, besides Google's proprietary artifacts, AppSheet integrates with third-party vendors like AWS, Azure SQL, or Salesforce. On a related note, using spreadsheets as model artifacts has also been explored by model-centric solutions like Epsilon [28], which offers a driver for spreadsheets [16] with a similar mapping approach.

Kissflow [25] is a web-based low-code platform specializing in applications that track entity lifecycles with business processes. Kissflow features a workflow language that integrates with enterprise tools like SAP and Salesforce. Moreover, it offers an AI copilot to assist during the development of workflows. The platform is highly customizable, allowing users to define roles, forms, pages, navigation, analytics, and integrations. The platform also features a proprietary expression declarative language to define application behavior.

Salesforce is a customer relationship management (CRM) SaaS to manage the full lifecycle of customer interactions, e.g., from receiving an order to its process and delivery. Its selling point is automating processes to avoid manual errors. Their low-code platform is **Salesforce Platform** [49], which builds atop Salesforce's CRM capabilities. The platform allows linking pages to data types and configuring permissions and applications. Pages defined within the tool may include dashboards that display data in tables and charts.

Appian [2] is a web-based low-code platform. It allows managing *record types* and *process models* that integrate data from databases, Salesforce, or web services. The platform features widgets to manage entities, as well as workflows for business automation. The platform is flexible with security, allowing fine-grained role and data access management.

Many low-code platforms rely on graph-like workflows to manage behavior. **Caspio** [6] proposes a different approach, whereby workflows are defined using the MIT App Inventor.[6] In particular, workflow nodes are SQL-based, providing a Scratch-like interface for building applications.[7] The platform supports building reports, forms, and charts directly from SQL tables.

Unlike other low-code solutions, UGROUND aims to produce Semantic Digital Twins [45]. These are developed following an approach based on a self-referential, ontology-driven architecture, enabling continuous adaptation without the need for bespoke software redevelopment [11], i.e., so-called *no-code*. The company covers several domains, from banking to insurance and human resources. The development of these platforms involves **ROSE** (Recursive Ontology Semantic Engineering) [11], both a methodology and a model-driven engine. In ROSE, applications are structured through semantic modeling, while *JCORE*, a dedicated general-purpose language, defines behavioral logic. The company has developed a comprehensive catalog of reusable functionalities and visual components, continuously expanding its internal JCORE library. Thanks to the reliance on a model-driven architecture, UGROUND can reuse many components across domains and applications, accelerating the development process.

Finally, there are few academic results on the intersection between MDE and full-fledged low-code platforms. Such an example is **BESSER** [1], an MDE-based low-code platform that relies on UML for structural modeling and on template-based code generators for execution. BESSER is still in development but, as our running example will show, a standard general-purpose domain modeling language (like UML class diagrams) is not the optimal choice to describe low-code applications. Instead, we argue that such a language needs to consider primitives specific to low-code, like platforms, users, roles, and resources. Similarly, code generation alone significantly limits the production of low-code applications. These generally would benefit from having available other services including model transformations or powerful model visualizations [32] that can be orchestrated and executed into the defined low-code application itself. Our workflow language, PLATFLOW, precisely follows this aim.

2.2 Workflows

Workflow languages permit describing processes in terms of resources, tasks, and decision blocks [61]. Workflows are often represented with diagrams composed of nodes and con-

[5] https://adaptivecards.io.

[6] https://appinventor.mit.edu.

[7] https://scratch.mit.edu.

nectors to facilitate their comprehension. They can be used for a variety of purposes and domains, from machine learning and data processing to business processes and protocol description. In this section, we review four broad workflow categories according to their purpose: process modeling, MDE development, MDE in the cloud, and low-code development.

2.2.1 Workflows for process modeling

Several workflow languages have been proposed for modeling processes. The most notable example is the Business Process Model and Notation (BPMN) [38], a standard to specify business artifacts behavior. Another example is the Software & Systems Process Engineering Metamodel (SPEM) [39], which relates processes, roles, and work products in an ordered manner. To facilitate further the definition of business processes, works like [46] define workflow patterns, which, in the style of '*The Gang of Four*' [18], correspond to recurrent structures that emerge in workflows, and may consider diverse aspects of the process such as control flow, data flow, or resource management.

2.2.2 Workflows for MDE development

MDE has typically taken place in dedicated editors (IDEs) for programming with facilities for model management. As such, these environments have stimulated the development of workflow languages. Some languages focus on very specific use cases, such as transformation chains [42, 52]. Others allow for general model management tasks, such as ModelFlow [50, 51], the workflow language within AToMPM [17]; or the Modeling Workflow Engine (MWE) [57], for Eclipse modeling components. In some cases, the traceability of the transformations in workflows is preserved for a better understanding and for enabling validation [26].

2.2.3 Workflows for MDE in the cloud

Some workflow languages have been designed to perform MDE activities in the cloud, while not being integrated within low-code environments. For example, MDEForgeWL [21] is a workflow language to orchestrate model management services, and MDEForge-Search [22] includes a query language to retrieve MDE artifacts from a distributed cloud-based model repository.

2.2.4 Workflows for low-code development

Most of the low-code platforms overviewed in Sect. 2.1 use workflows in different ways. For instance, workflows in Mendix feature loops, decisions, and recursive actions, in the same way as OutSystems. This latter platform, in particular, distinguishes between client and server actions. In Zoho, the focus is on event-driven workflows with basic logic (e.g., decisions or delays). Microsoft Power Apps integrates with Power Automate, also from Microsoft, providing robotic process automation (RPA) unlike other low-code platforms. Many LCDPs make use of expression languages, either declarative (e.g., Power Fx in Power Apps or Google AppSheet) or imperative (e.g., Deluge in Zoho Creator).

2.3 Gap analysis

The approach of Dandelion+ lies in the intersection between low-code development, MDE, and workflows. Our goal is to investigate whether a symbiosis of techniques from these paradigms can overcome the limitations existing in current low-code platforms.[8]

MDE stands out for its flexibility in defining solutions for specific domains based on the use of models. However, MDE workflows are typically tied to concrete IDEs, featuring facilities for them, but impeding their use in other environments such as the cloud. MDE workflows for the cloud are typically ports from other domain-specific workflows, thus still not being appropriate for low-code development, since they lack primitives and integration with low-code platforms. This is also a limitation of generic workflow languages for process modeling. As a result, no workflow language enables low-code behavior development in the cloud while following a model-driven approach—Dandelion+ addresses this need with PLATFLOW.

Regarding low-code platforms, Table 1 compares current LCDPs with our approach, Dandelion+, in terms of the offered capabilities for the following aspects of applications: model-driven features, workflows to define application behavior, access control rules, and external integrations.

Most LCDPs in Table 1 lack MDE features. In particular, almost no LCDP supports the definition of custom application generators. The only exceptions are Power Apps and Mendix, which support PDF and HTML generation; BESSER, which lacks support for key low-code aspects like users or roles; and our tool, Dandelion+, which offers tech-agnostic code generation with EGL/EGX. Thus, LCDPs generally follow the working scheme illustrated in Fig. 1a, leaving citizen developers with few options for customizing their applications for specific domains. LCDPs rely on built-in application generators that produce similar applications, with little room for further customization.

Regarding workflows, LCDPs typically allow the encoding of behavior using dedicated nodes, such as for entity

[8] Note that our goal is *not* building a full-blown professional LCDP. The scope of this paper is limited to the investigation of bringing MDE techniques to the low-code world and assessing their benefits.

Table 1 Features for application configuration of the leading commercial and academic LCDPs

	Model-driven features					
Platform	Custom code gen.	Meta-modeling	Model transf.	Workflows	Access Control	Integrations
OutSystems [40]	○	◒ relational DB schema with visual editor	◒ to adapt data to populate fields of widgets	Divided into server- and client-side; supports actions like message, compute data, refresh data, conditionals, loops, etc.	Application, data, and screen levels; fine-grained control	SOAP, REST, SAP
Mendix [34]	◒ PDF, HTML	◒ object-oriented schema with visual editor	○	Microflows for object activities, REST services, Java actions, ML activities, decisions, and merges	CRUD permissions for types with XPath expression constraints	REST, SAP, and other external services
Zoho Creator [66]	○	○	○	Workflow nodes include apps (integrations), logic (IF, decision, set variables), and integrations	Filter pages by role, visibility, and read-only status	500+ integrations (Salesforce, GitHub, Zoho CRM, Slack)
Power Apps [35]	◒ PDF, HTML	◒ relational DB schema	◒ to select (custom) columns of a data entity	RPA and DPA for automating user interactions and backend workflows; executes workflows on events and schedules	Ownership models with permissions (full, reference, collaborate, private, etc.)	Power BI, Power Automate, AI Builder, Microsoft Dataverse
AppSheet [20]	○	◒ relational DB schema	○	Bots define workflows; simple tasks like email, notifications, and webhooks	Role-based filters on fields and attributes with expression language	Google Sheets, Office 365, Salesforce, Cloud SQL, Dropbox
Kissflow [25]	○	○	○	Business process modeling using drag-and-drop workflows with forms and integrations	CRUD control per type, form roles for manage/read-only/edit	SAP, Salesforce

Table 1 continued

Platform	*Model-driven features*			Workflows	Access Control	Integrations
	Custom code gen.	Meta-modeling	Model transf.			
Salesforce [49]	○	◒ relational DB schema with visual editor	◒ derived fields	Workflow includes decisions, loops, create/update elements, and subflows	Fine-grained control per page, type, and role	Salesforce ecosystem (Sales Cloud, Marketing Cloud, etc.)
Appian [2]	◒ HTML, PDF, Word	◒ relational DB schema with visual editor	◒ derived fields	Process models with events, tasks, automation, data services, APIs, and document generation	Complex role and object-based permissions with granular access control	Salesforce, databases, REST APIs
Caspio [6]	○	◒ relational DB schema	○	Scratch-like puzzle-piece workflows using SQL-based nodes (INSERT, UPDATE) with limited automation	Limited access control through apps	Direct SQL access
ROSE [11]	◒ imperative JCORE code	● multi-level modeling: ROSE with visual editor	◒ imperative JCORE code	Business process models (BPMs), to model the lifecycle of entities; finite state machines	Managed at the entity level, has roles	SOAP, REST, SAP, DBs (Oracle, Elastic, Mongo DB, etc.)
BESSER [1]	● tech-agnostic, with Jinja templates	● full modeling: B-UML with PlantUML editor	○	–	–	–
Dandelion+	● tech-agnostic, with EGL/EGX	● full modeling: Dandelion format with visual editor	● tech-agnostic, with EOL and ETL	Workflows can be chained and executed within the platform, offering PLATFLOW for orchestrating both MDE and low-code tasks	Complex, platform-specific access control tied to MDE models	Third-party APIs, format importers, Eclipse Epsilon drivers

Legend: ● full support, ◒ partial support, ○ no support

management (CRUD operations) and connecting to third-party services. Workflow processes may also call other workflows. Access control is often specified through expression languages or forms, allowing permissions and roles to be defined for both citizen developers (who can access the development environment to modify the application) and/or for end users of the runtime applications. Most LCDPs also support integration with external services, though the scope of integrations varies between platforms.

Overall, Table 1 shows that all LCDPs (but BESSER) provide mechanisms to specify and execute workflows, to define access control policies, and to integrate external services within the LCDP operation. Even if the expressiveness of the workflow primitives, the granularity and customizability of the access control, and the integrability possibilities largely vary among LCDPs, it is clear that these three aspects are key to any successful LCDP today. In this respect, as we discussed at the beginning of Sect. 2.1, MDE solutions typically run on desktop computers and do not provide first-class support for these aspects, which hinders their applicability for low-code development.

Hence, we identify two gaps in the state of the art:

1. On the one hand, industrial LCDPs lack MDE features, especially code generation facilities. As a result, they become unsuitable for the scenario depicted in Fig. 1b.
2. On the other hand, academic LCDPs (i.e., BESSER) mostly port MDE concepts to the cloud. Currently, they lack support for low-code concepts (e.g., platforms, users, or files), integration via APIs, access control, and workflow facilities.

In the following, we describe our approach to fill these gaps with Dandelion+ and PLATFLOW.

3 Running example

In this section, we present our running example and outline the limitations encountered in its development when using current low-code approaches.

The domain of our running example is *learning management systems* (LMSs)—platforms in which academic activities are carried out. Some of the most prominent examples include Moodle[9] and Canvas.[10] These platforms typically feature activities and learning tools (e.g., quizzes and assignments), interactive spaces for lecturers and students (e.g., announcements or discussion forums), and academic management facilities (e.g., uploading and downloading documents, grading quizzes, or conducting surveys) [36].

[9] https://moodle.org.

[10] https://www.instructure.com/canvas.

We will employ '**Mudel**,' a fictitious LMS, as a concrete LMS for our running example. Mudel supports common features of LMSs, including managing courses where to post announcements and link learning material, creating and submitting assignments, and elaborating and answering quizzes.

In Fig. 2, we propose a possible class diagram that captures the structure of Mudel. The central concept is the Course, in which Users can be enrolled with different roles, namely professors, students, teaching assistants (TAs), and external personnel. The main page of each course is divided into Sections, which may contain Announcements posted by professors and may link pages and assignments designed for the course (i.e., AssignmentLinks and PageLinks). Pages contain text (TextContent) as well as other media such as links to external webpages (LinkContent) or file attachments (FileContent), making them appropriate for lectures or notes. Professors can propose exercises to students through Assignments, and students can submit their work via an AssignmentSubmission, which may enclose file attachments. Professors can also create Quizzes with Questions, each with several QuizAlternatives in turn. Students can answer them via QuizSubmissions, which choose one alternative per question. Finally, sections, assignments, and quizzes can be hidden or made visible with the isVisible property.

While this class diagram adequately captures the essence of the domain of LMSs, we identify the following issues in standard domain modeling languages (e.g., UML class diagrams) when it comes to their use within LCDPs:

I1. *Access control is insufficiently modeled.* Low-code platforms need access control to restrict the actions that users can perform. However, although Roles are modeled in Mudel, their permissions are not. As a consequence, access control is insufficiently described and cannot be enforced. This leads to undesired scenarios, such as students creating courses, or modifying submissions of peer students. Note that typical domain modeling languages (e.g., Ecore or UML class diagrams) do not consider access control. Even the use of integrity constraint languages such as OCL [37] would not solve the issue or lead to ad-hoc solutions. Therefore, mechanisms that reify roles and permissions [7] to drive and permit the enforcement of access control on the elements defined on a domain model would be needed.

I2. *Ownership is not modeled.* In regular modeling languages, ownership is not modeled; i.e., who creates an entity is not relevant. However, ownership is crucial in some domains developed with low-code platforms. The reason being that users themselves may be responsible for managing entities, and therefore it is important to

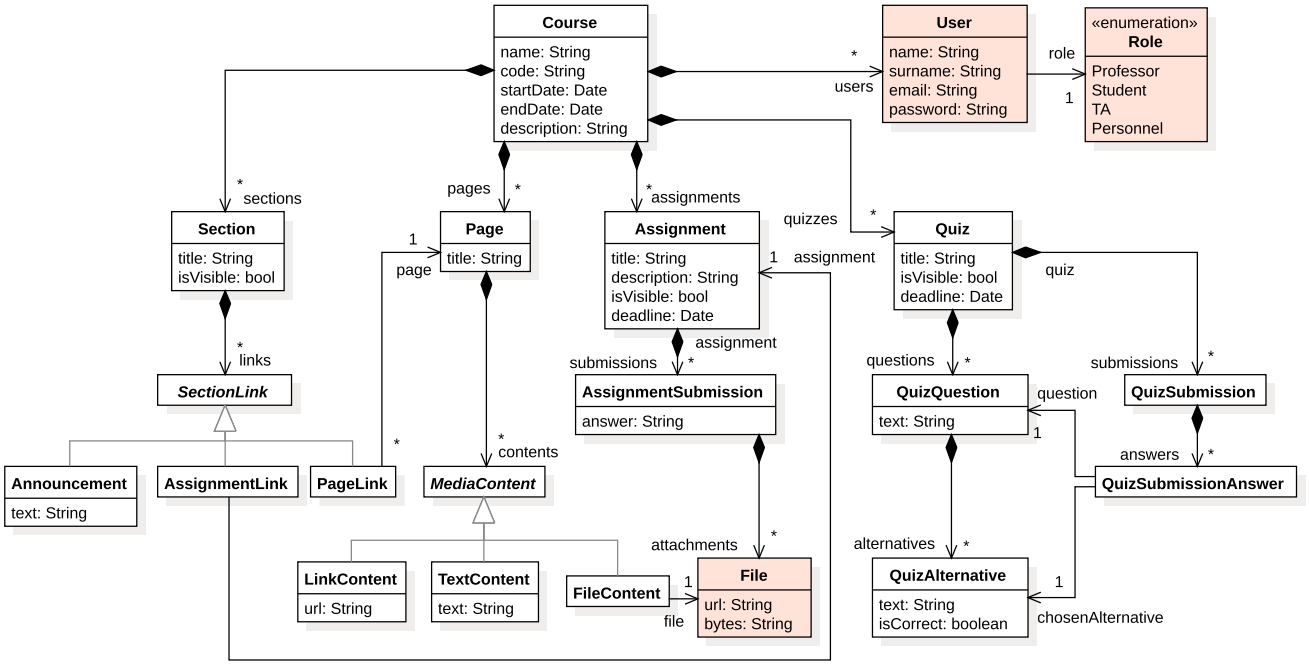


Fig. 2 Class diagram of *Mudel*, a fictitious learning management system. Problematic classifiers are highlighted

discern who owns a given entity. In the case of Mudel, modeling the ownership of submissions is necessary to know who made them and to limit students' access to only their own. Therefore, platforms must keep track of the ownership of the entities defined therein, expose programmatic interfaces to manage entities' ownership, and offer access control mechanisms that exploit this information.

For clarification, we could improve the diagram with associations between User and some classes to model ownership (e.g., the professor creating a Course or the student submitting an AssignmentSubmission). However, these ownership relations and the involved classes (e.g., User or Role) would require a dedicated treatment in the modeling language to convey their special semantics to the low-code platform.

I3. *Equivalent platform- and domain-specific entities are not bridged appropriately.* LCDPs manage a constellation of entities, including pages, users, data schemas, and the data itself. These entities can be divided, in broad terms, into two categories: *platform-specific* and *domain-specific*. The former includes pages and configurations about the platform, while the latter includes entities that belong to the domain of the DSLs (e.g., Course or Quiz in Mudel). However, these sets are not disjoint: the platform manages some entities, which DSLs may also include to relate them with other domain concepts. This is the case of User, Role, and File: while these concepts are clearly platform-specific (i.e., they are central to the functioning of any low-code platform), they may also belong to some DSLs. For instance, in Mudel, assignment submissions may attach files that the platform should handle especially and appropriately. However, as it is, the class diagram in Fig. 2 does not account for this connection. In particular, users and roles should not be secluded within DSL definitions, as the platform must manage them for authentication (i.e., logging in) and authorization (i.e., access control). Similarly, files can be part of the low-code platform definition itself as files with code snippets for its configuration. In conclusion, modeling languages for low-code development must provide mechanisms that bridge domain-specific entities with some platform-specific ones, such as users, roles, and files, avoiding entity duplication and ensuring a correct management.

I4. *Behavior is not modeled.* The running application consists of pages that typically contain forms and buttons that perform actions upon submitting or clicking on them. For instance, a student may post a question in a forum, and a professor may grade a quiz. However, the class diagram alone does not capture this information. Therefore, it is essential to provide mechanisms to model behavior, possibly through high-level workflows.

I5. *Applications are not modeled.* Besides access control, this (meta-)model is insufficient to generate a running LMS without assumptions. For instance, the LMS will contain non-obvious pages such as the one presenting quizzes together with their questions and alternatives.

Therefore, there is the need to model pages, to display different entities and other elements of the application.

Moreover, we identify the following issues when it comes to the development of Mudel in conventional LCDPs:

I6. *Facilities for data manipulation and artifact generation are insufficient.* As Table 1 shows, most LCDPs in the market support some type of (meta-)modeling. However, they lack mechanisms to manipulate the data they manage and to produce artifacts from it using model transformation and code generation, respectively. This directly refers to the scenario depicted in Fig. 1b. Mudel has several realizations of this scenario. For instance, professors may generate a static site from the sections of a course, generate code for other tools, or export quizzes using the *Moodle XML format*,[11] a Moodle-specific format for exchanging quizzes.

I7. *Importing/exporting mechanisms are not extensible.* LCDPs support different import/export mechanisms. For instance, some LCDPs can manipulate JSON documents, while others, like OutSystems, integrate with Excel spreadsheets. However, the support is limited given the lack of data manipulation facilities (I6). This disparity of capabilities can hinder the adoption of a particular LCDP and produce vendor lock-in. For example, this would preclude importing quizzes in Moodle XML format into the example platform. Therefore, it is necessary to provide extensible mechanisms for importing and exporting data.

While we have identified issues I1–I7 in the domain of LMSs, these are general enough to be extrapolated to other domains and are prone to impact the development in any LCDP. The following sections explore our approach to overcome these limitations using Mudel as an illustration.

4 Approach

In this section, we introduce our approach by outlining the guiding principles behind the design of Dandelion+ and providing an overview of its parts.

Dandelion+ is an evolution of Dandelion [31], a cloud-based graphical language workbench built upon MDE principles. Dandelion focused on three main aspects: (i) *model scalability*, leveraging a bespoke Elasticsearch [13] persistence layer to handle models containing hundreds of thousands of instances; (ii) *(meta-)model heterogeneity*, enabling a uniform representation of artifacts coming from different technologies; and (iii) *concrete syntaxes*, extending the designed languages with a visual notation usable from a web browser. Dandelion+ builds on Dandelion's persistence layer, thereby inheriting its high model scalability and heterogeneity. However, its main focus shifts toward providing a comprehensive application development infrastructure—from platform structure to behavior—to enable the creation of full-fledged low-code platforms.

In particular, Dandelion+ is based on the following principles:

P1. *Dandelion+ has a model-driven foundation.* As discussed in [59], the adoption of MDE techniques into low-code platforms brings benefits, including overcoming the issues of scalability (i.e., supporting many entities), openness (i.e., supporting different formats, enhancing interoperability, and preventing vendor lock-in), and heterogeneity (i.e., supporting artifacts expressed in different formats). Dandelion+ follows a model-driven approach to design DSLs that capture platform domains, as well as the models that are instances of these DSLs.

P2. *Dandelion+ has a uniform ecosystem with a reflective interface.* Low-code platforms work at different levels of abstraction, including platform definition (e.g., users, roles, or the platform itself), DSL definition (i.e., meta-models), and DSL usage (i.e., models). When developing complex applications, it is common to encounter scenarios that involve the whole modeling ecosystem and permeate all these levels of abstraction (e.g., dashboards).
Therefore, a *unified* interface that bridges all these levels is desirable. Moreover, applications may be *reflective*, i.e., they may modify the platform itself at runtime.
Dandelion+ achieves reflectiveness by explicitly modeling each aspect of the platform and enabling its runtime manipulation via model management languages. Technically, it relies on Epsilon [28] for model management, using the Epsilon Object Language (EOL) uniformly for all model operations. Another benefit of using Epsilon throughout the platform is persistence decoupling. LCDPs in the market have different persistence mechanisms, potentially yielding vendor lock-in. Although Dandelion+ provides an implementation in Elasticsearch [13], the persistence layer is handled by the Epsilon Model Connectivity Layer (EMC) [28]. This layer acts as an intermediary, making the platform persistence-agnostic and shielding entities and logic from underlying storage mechanisms.

P3. *Platforms are structured using a low-code specific linguistic meta-model.* Issues I1–I3 show that a standard domain modeling language (e.g., UML) is not expres-

[11] https://docs.moodle.org/405/en/Moodle_XML_format.

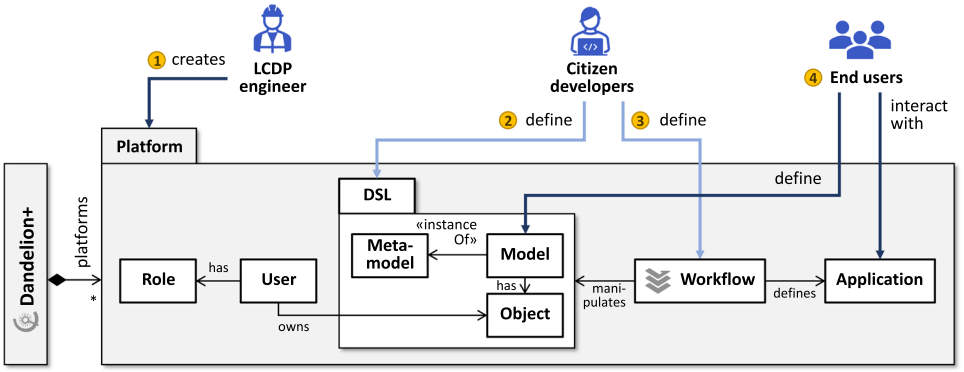


Fig. 3 Top-level structure of Dandelion+ and interactions with stakeholders

sive enough to handle key aspects of low-code platforms such as access control, ownership, and bridging domain and platform entities. Dandelion+ addresses these issues by relying on a linguistic meta-model tailored for low-code platforms. In particular, the structure of the platform is managed through a *platform* meta-model, ownership and entity bridging are managed through a bespoke *domain modeling* meta-model, and access control is managed through a *roles* meta-model.

P4. *Application behavior is implemented using workflows.* As stated in issues I4 and I5, defining applications and their behavior is essential in any non-trivial low-code platform. In Dandelion+, applications are modeled using an *applications* meta-model, and their behavior is described using PLATFLOW, a platform-aware graphical language to define and configure domain-specific applications using workflows. In order to improve the experience of citizen developers, workflow development takes place on the browser, sparing the need for installing desktop applications or plugins. Workflows include nodes for different operations, like model transformation, code generation, and integration with external services. PLATFLOW works on top of Epsilon (cf. principle P2) to shield the implementation from technology specificities. The components of PLATFLOW enable data manipulation in a systematic and controlled way (I6). To address I7, PLATFLOW allows importing and exporting data in different formats via serializers (drivers) that leverage the Epsilon Model Connectivity Layer (EMC). Moreover, Dandelion+'s domain modeling meta-model allows the homogenization of data in different formats, thereby ensuring interoperability.

Figure 3 presents a bird's-eye view of our approach. Every instance of Dandelion+ contains several *platforms*, which are isolated development environments. This allows their exploitation by different tenants without interfering with each other. Each platform manages its *users*, whose permissions are determined by their *roles*. Platforms also define *DSLs*, which can be instantiated in *models*, just like in standard MDE. Furthermore, platforms can define complex *applications*, which are driven by *workflows* that can either manipulate or make use of DSLs and models, as well as the entire platform.

Dandelion+ supports three main stakeholders, who incrementally build the low-code platforms. The steps are numbered in the figure. First, the LCDP engineer creates a platform (1). Automatically, an *Administration* role is created, together with an administrator user, along with predefined components for its working. Additionally, the LCDP engineer can register more roles and users. Then, those citizen developers registered by the LCDP engineer can authenticate to the platform and create domain-specific languages (2). To do so, they can register meta-models into the platform either using a textual syntax or via an integrated graphical meta-model editor. Then, citizen developers can create workflows (3). Finally, end users can either modify models manually or interact with applications (4). The experience of both citizen developers and end users is shaped by their assigned roles.

In the following, we describe the two main ingredients of low-code development with Dandelion+. In Sect. 5, we present our approach to define the structure of low-code platforms (i.e., platforms, DSLs, roles), whereas in Sect. 6, we detail the infusion of behavior into applications using a workflow language.

5 Structure of low-code platforms

This section presents our approach to designing the structure of low-code platforms. To do so, we formalize the entities encompassing low-code platforms and their relationships following a model-driven approach in a *linguistic meta-model* that satisfies the principles detailed in Sect. 4. The meta-model is broken down into packages, which are examined in the next subsections. In particular, the core of this meta-model is the *platform* package (cf. Sect. 5.1) which uses the rest of the packages: the *domain modeling* package (cf. Sect. 5.2) which allows defining DSLs within platforms; and the

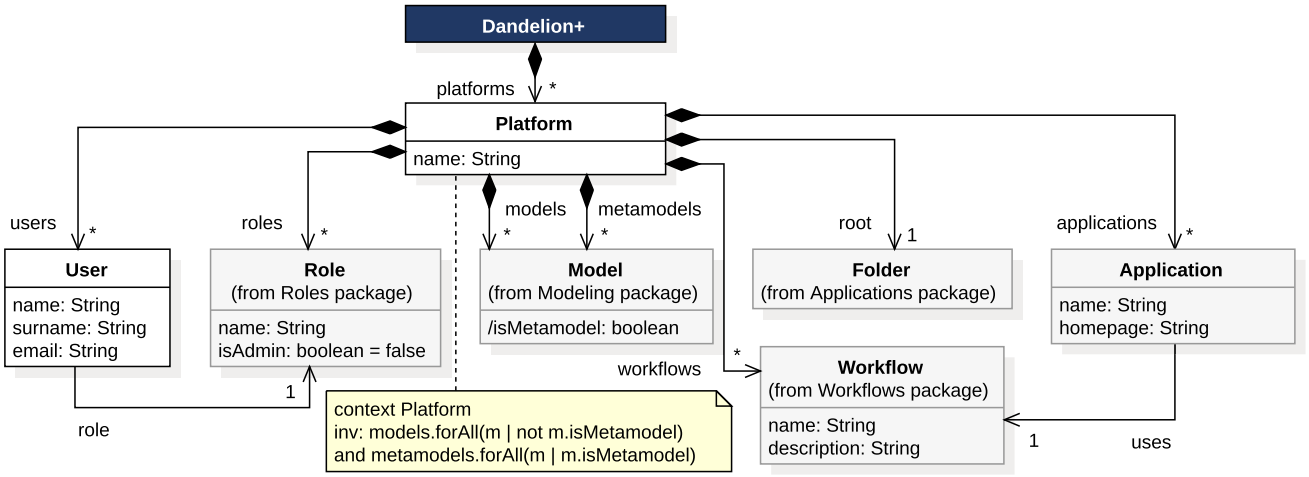


Fig. 4 The platform package

roles package (cf. Sect. 5.3) which permits defining user permissions.

5.1 Platform meta-model

The *platform* is the topmost concept in Dandelion+. Everything in Dandelion+ is contained within a platform. As such, platforms provide independent and isolated development environments that coexist under one instance of Dandelion+.

Figure 4 presents the platform package of Dandelion+'s linguistic meta-model. Platforms have a name and encompass several concepts. In particular, platform stakeholders are reified as Users, allowing their authentication therein. The experience of these users within the platform is shaped by their assigned Role. These very same users can define and operate with DSLs, whose construction platforms support through meta-models and their instances (both can be represented using Models, as explained later). DSLs can be managed in a low-code fashion using Workflows, which can be promoted to full-blown Applications available for the end users. Finally, platforms can store Files, which can contain executable code (e.g., for defining specific parts of workflows) or any other type of static content, such as images or documents. For this reason, platforms point to the root Folder of its associated file system.

5.2 Domain modeling meta-model

Dandelion+ uses a domain modeling meta-model to represent domains and define DSLs. The definition of these DSLs is specified in meta-models, and their instances are represented in models. This meta-model is a continuation of the "harmonizing meta-model" designed for Dandelion [31], which we have adapted and augmented to cater for the requirements of operational low-code platforms.

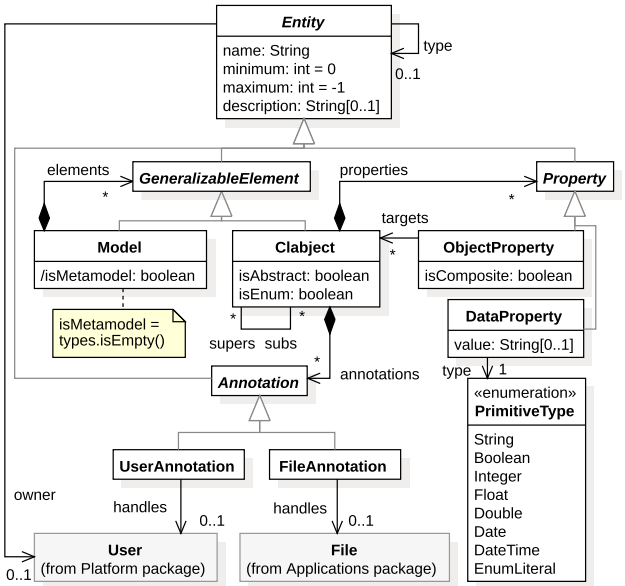


Fig. 5 The domain modeling package

Figure 5 presents the domain modeling meta-model. The central concepts are Model and Clabject. On the one hand, Models reify both models and meta-models. Their elements are objects and meta-classes, respectively. On the other hand, Clabjects[12] are used to represent these objects and meta-classes, populating or defining properties, respectively [3]. Properties can either be encoded through DataProperty, for primitive types, or ObjectProperty, for associations or containments. Models, clabjects, and properties are all subclasses of Entity, which has a name, may have a description, and may be typed. Entities also define minimums and maximums, whose semantics depend on the concrete realiza-

[12] This term is taken from multi-level modeling to refer to an entity that can play the role of both a *cla*ss and an o*bject*.

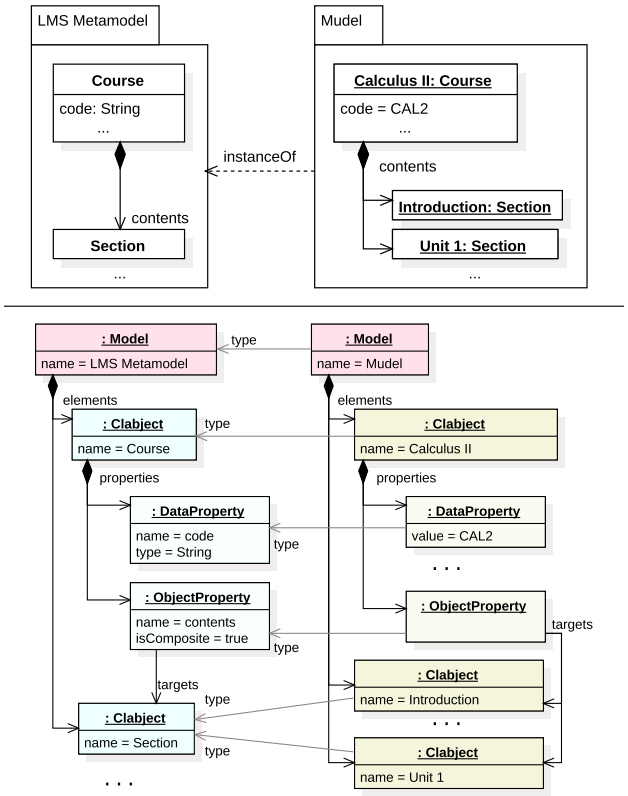


Fig. 6 Expressing a meta-model and one of its instances using the modeling meta-model

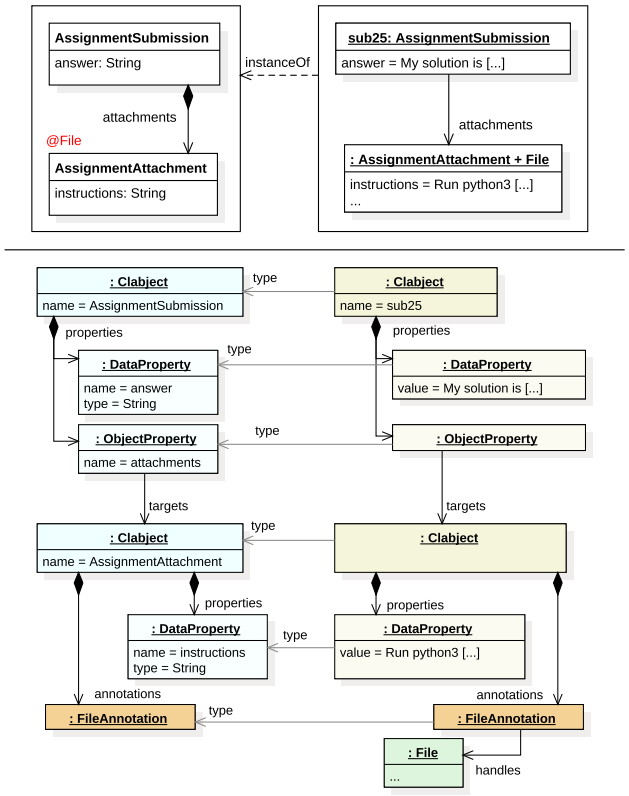


Fig. 7 Supporting files in a meta-model using annotations from the domain modeling meta-model

tion: in clabjects, they constrain their number of instances, whereas, in properties, they define multiplicities.

Figure 6 exemplifies how to express a part of Mudel's meta-model in Fig. 2 and how to instantiate it using the modeling meta-model. Specifically, it represents the LMS metamodel by defining concepts like Course and Section, as well as an instance named *Mudel* of said meta-model that contains a Calculus II course with some sections. Please note that instantiation is symmetrical: Models (meta-models/models) contain Clabjects (meta-classes/objects) which, in turn, define/populate Properties. Type conformance is preserved thanks to the Entity.type relationship.

One of the issues we identified in the running example is that users and files must be modeled differently to be usable from a platform standpoint while being expressed normally in the meta-model as meta-classes. Our approach is to decorate the impacted meta-classes with Annotations: we consider the case of users and files with UserAnnotations and FileAnnotations, respectively. Both meta-classes and objects can be annotated. In particular, objects of an annotated meta-class point to the concept (i.e., user or file) they handle.

Figure 7 demonstrates how to apply these annotations to represent AssignmentSubmissions and Files from the running example. The meta-class AssignmentSubmission is represented as a Clabject, with a data property for its answer and an object property for the attachments. Files, instead, are handled differently. In Dandelion+, it makes sense to promote them to a meta-class (i.e., AssignmentAttachment) annotated with a FileAnnotation. The objects that instantiate this meta-class follow the regular procedure for instantiation, but are also annotated with FileAnnotations that point to files. With this approach, the expressivity of the initial class diagram is maintained (i.e., files are part of the domain language), the platform can handle files correctly, and it allows defining more properties over users or files (e.g., AssignmentAttachment.instructions).

5.3 Roles meta-model

Stakeholders participate in low-code platforms with different goals and scopes of action. Therefore, implementing access control is paramount. Dandelion+ manages this through *roles*.

Figure 8 presents the roles meta-model. Each platform defines a series of Roles, which are assigned to Users. Roles have a name and can be administrators, who are granted full control to modify the platform, including managing roles and users. Each role is associated with a series of applications that constitute their menu in the browser. A role declares Permissions, which determine whether the creation,

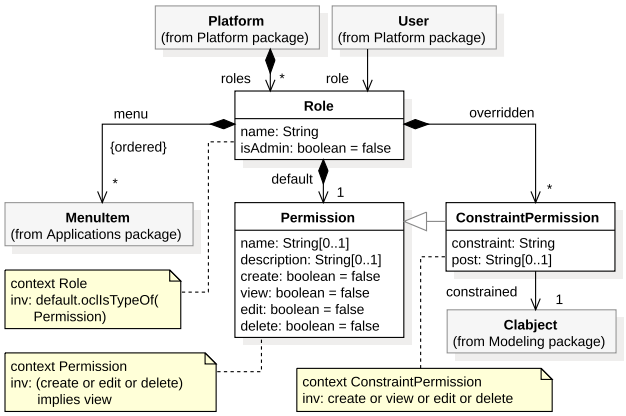


Fig. 8 The roles package

query, edition, or deletion of entities is allowed (i.e., create, view, edit, and delete). Specifically, every role must define a default permission, which is applied to all the meta-classes in the system. This behavior can be superseded via ConstraintPermissions through Role.overridden. In particular, ConstraintPermissions are linked to the meta-class they target (constrained), and define two fields: constraint and post. The former expects an EOL Boolean predicate indicating whether the permission should be applied. That is, whenever an object of type constrained is found, constraint determines whether the object is made available or not. The latter expects any EOL code and allows modifying objects immediately after retrieval. This mechanism can be leveraged for convenience to perform transient, local model transformations. In both fields, variables 'this' and 'currentUser' are available, referring to the object being queried and the user performing the action, respectively. If a clabject type is pointed by several overridden ConstraintPermissions, Dandelion+ only grants those permissions (i.e., create, view, edit, delete) for which no constraint evaluates to false.

Table 2 summarizes Mudel's roles and their permissions following this meta-model. In particular, professors can do everything except manage students' submissions; students can only view contents on the platform, as well as create and edit their submissions; TAs can modify courses and their contents; and personnel has access to all the information therein but in a read-only manner.

However, this permission specification is too coarse-grained; the domain of Mudel is more restricted. For instance, we identify the additional restrictions gathered in Table 3 for the student role. They restrict an action (in italics) over a meta-class given a condition (i.e., a predicate). These more refined restrictions can be modeled using ConstraintPermission: the permission is enforced if and only if the constraint is satisfied. Moreover, the post field can be used to modify the object in place after retrieval.

Table 2 Roles and permissions of Mudel

Role	Create	View	Edit	Delete
Professor · *(Default)*	●	●	●	●
· AssignmentSubmission	○	●	○	○
· QuizSubmission	○	●	○	○
· QuizSubmissionAnswer	○	●	○	○
TA · *(Default)*	○	●	○	○
· Course	○	●	●	○
· Section	●	●	●	●
· Page	●	●	●	●
· AssignmentSubmission	○	○	○	○
· QuizSubmission	○	○	○	○
· QuizSubmissionAnswer	○	○	○	○
Student · *(Default)*	○	●	○	○
· AssignmentSubmission	●	●	●	○
· QuizSubmission	●	●	●	○
· QuizSubmissionAnswer	●	●	●	○
Personnel · *(Default)*	○	●	○	○

Legend: ● granted, ○ not granted

Table 3 Restrictions of the student role in Mudel

#	Meta-class	Condition ("can only be...")
R1	Section	*viewed* iff they are visible
R2	Assignment	*viewed* iff they are visible
R3	Quiz	*viewed* iff they are visible
R4	QuizAlternative	*viewed* with isCorrect hidden
R5	AssignmentSub..	*viewed* if the object is owned
R6	QuizSubmission	*viewed* if the object is owned
R7	QuizAnswer	*viewed* if the object is owned
R8	AssignmentSub..	*edited* before the deadline
R9	QuizSubmission	*edited* before the deadline
R10	QuizS...Answer	*edited* before the deadline
R11	AssignmentSub..	*created* before the deadline
R12	QuizSubmission	*created* before the deadline
R13	QuizS...Answer	*created* before the deadline

Figure 9 shows the implementation of some restrictions for the student role using the roles meta-model. In the model, the default permission enables to view everything. Then, ConstraintPermissions override this general permission for particular clabject types. In particular, R1 enables to view only available Sections (those for which this.isVisible is true); R5 restricts the view to owned AssignmentSubmissions; R8 enables editing within the deadline; and R4 hides the correct alternatives (i.e., the solutions of the assignment) via a post attribute. Please note that these modifications are not persisted, but computed to create a view.

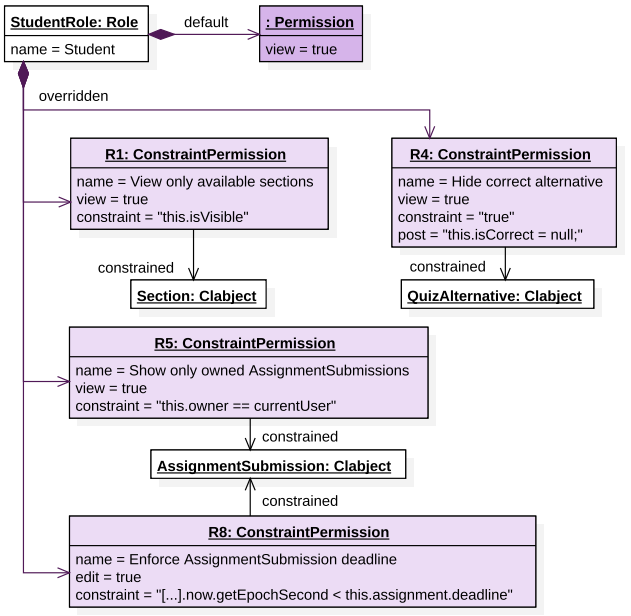


Fig. 9 Implementing permissions for the student role using the roles meta-model

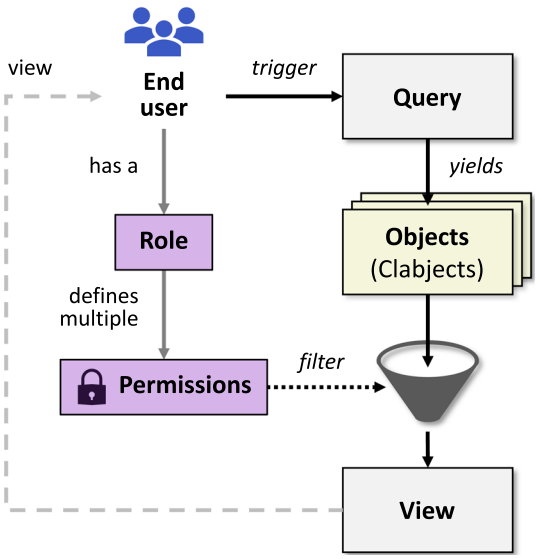


Fig. 10 Views processing according to the permissions assigned to the current user's role

Finally, Fig. 10 illustrates the integration of roles within Dandelion+. First, the end users interact with the platform and trigger the query of some data. Upon resolution, the query yields a set of objects (clabjects) filtered according to the user's role and permissions. In case of inconsistency, the most restrictive permission applies. Moreover, the filtered objects shape the view presented to the end user. Finally, recall that ConstraintPermission.post modifications must be local (e.g., modify the value of an attribute) and are not persisted. This way, the view-update problem from databases [4] is circumvented.

6 Behavior of low-code platforms

The meta-models presented in the previous section enable the definition of the structure of low-code platforms, from the definition of DSLs and their instantiations to access control policies. However, although this approach is sufficient for simple use cases (such as the CRUD dashboards offered by Google AppSheet), rich applications typically depend on components with an associated *behavior*. That is, they require executing a set of operations (i.e., algorithms) to fulfill their operational requirements. For instance, reports need to query, analyze, and aggregate data for their visualization; and forms may perform validations over their input fields before storing the provided data into new entities.

This section presents the approach of Dandelion+ to model behavior, which includes *workflows* and *applications*. First, we introduce PLATFLOW, our language for defining workflows in low-code platforms, in 6.1, and explain the execution of the workflows defined in this language in 6.2. Next, we present some exemplary workflows in 6.3 for the Mudel platform. Finally, we present the meta-model for deploying applications on Dandelion+ using workflows in 6.4.

6.1 PlatFlow

PlatFlow is a workflow language tailored for specifying and executing behavior within low-code platforms. In the following, we present its design rationale (6.1.1), the structure of the workflows defined with it (6.1.2), and a catalog of its available nodes (6.1.3).

6.1.1 Design rationale

PLATFLOW is founded on **domain adequacy** and **technological decoupling** for workflow execution.

On the one hand, PLATFLOW is meant to define behavior directly in the browser. To achieve this, we have created a drag-and-drop graphical editor that features nodes that interact with the platform where workflows are executed (e.g., managing entities). Additionally, the editor is designed to be **platform-aware**: many nodes include dropdowns and search boxes to quickly locate entities within the current platform, facilitating workflow development.

On the other hand, PLATFLOW's execution is decoupled from the persistence layer used by Dandelion+ (or the LCDP workbench it integrates with). This is achieved thanks to a heavy reliance on Epsilon [28], which is decoupled from the peculiarities of the underlying technology by the Epsilon Model Connectivity Layer (EMC).

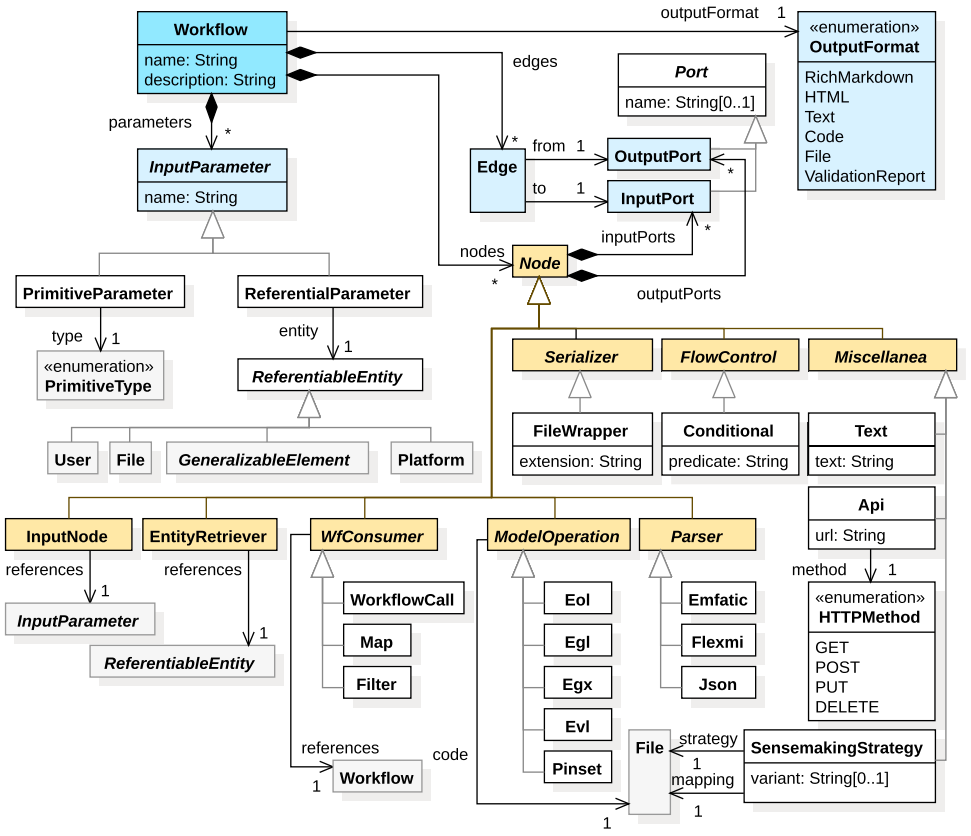


Fig. 11 Workflows package

6.1.2 Workflow structure

A **workflow** is an executable entity that, given a set of inputs passed as parameters, produces a result. Workflows consist of an exposed **interface** and **implementation logic**. As with the rest of Dandelion+, workflows in PLATFLOW are developed following a model-driven approach. Figure 11 presents the meta-model that captures their structure.

Regarding their interface, Workflows have a name, a description, may define input parameters, and have an output format. First, InputParameters have a name and are typed. In particular, they can have a primitive type (i.e., PrimitiveParameter, typed by a PrimitiveType from Fig. 5) or may reference platform-specific entities, such as users, files, models, or meta-models (ReferentialParameter). Second, OutputFormat specifies how to render the output of the workflow, when executed. It can be textual, including plain text, HTML, Markdown, or code snippets; or structured, including model objects (as returned by EOL or other Epsilon languages), files, and validation reports from EVL.

Regarding their *implementation logic*, workflows consist of Nodes connected by Edges. In order to correctly redirect the information between nodes, these expose input and output Ports, which can be named. Most nodes have one or two input ports and an output port. Moreover, some nodes have extensible input ports, allowing the creation or removal of input ports at will. Appendix A attaches the constraints of the meta-model, including those regarding the number of input and output ports of each node.

From a computation standpoint, workflows are similar to regular software programming functions. However, beyond regular programming, workflow nodes permit defining high-level operations over platform entities via a graphical editor, reducing the need for writing code.

6.1.3 Workflow nodes

PLATFLOW supports the following concepts:

- *Input nodes.* This type of nodes is used to connect the value of an input parameter to other nodes of the workflow.
- *Entity retrievers.* EntityRetriever nodes load entities within the current platform for their subsequent manipulation in the workflow. The entities that can be managed include users, files, models, meta-models, objects, meta-classes, and the current platform. There are as many nodes as entity types, and each node has a dropdown to find the desired entity (except for the current platform retriever node).
- *Workflow consumers.* These nodes leverage other workflows for their execution, enabling workflow reuse

through the references association. We distinguish three types:

  - WorkflowCall. This node executes another workflow defined in the system. The input parameters of the workflow are passed via named input ports. Therefore, the node requires a one-to-one correspondence between the input ports of the node and the parameters of the referenced workflow. That is, they must match in quantity and by name. The node's output is the result of executing the referenced workflow with the provided inputs.
  - Map. This node applies the referenced workflow to each element of an input collection, yielding a new collection of transformed elements. The referenced workflow must have exactly one input parameter, and the input collection is the flattened union of all values received through the input ports. The node outputs a collection containing the results of applying the referenced workflow to each input element.
  - Filter. This node is similar to Map, but instead of transforming elements, it selectively retains those satisfying a predicate. The referenced workflow must have exactly one input parameter and return a Boolean value. The input collection is constructed the same way as in Map, and the output collection contains only the elements for which the referenced workflow evaluates to true.

  Overall, the Map and Filter nodes act as higher-order functions, enabling loop-like behavior in workflows.
- *Model operations.* These nodes wrap Epsilon [41] language calls to perform model operations. All of them accept an arbitrary number of input ports, which are promoted to context variables within the scripts, and require their code to be specified in a File. Following this approach, code snippets can be shared among different workflows. First, Eol executes Epsilon Object Language [29] code, implementing arbitrary operations over any entity. Second, Egl and Egx make use of the Epsilon Generation Language and the EGL Co-Ordination Language [43], respectively, to generate artifacts from templates: Egl populates only one template, while Egx may produce multiple artifacts. Finally, Pinset [9] generates tabular structures out of models, and Evl validates models against EVL scripts [28], generating a summary of the violated constraints.
- *Parsers.* Parser nodes make use of existing parsers and Epsilon drivers[13] to structure textual information. First, the Emfatic parser consumes textual descriptions of meta-models in Emfatic and outputs a Dandelion-specific meta-model. Second, the Flexmi[14] parser consumes a meta-model and a FLEXMI [27] description of a model to load it into the workflow. Finally, the Json parser produces a queryable in-memory model from a JSON structure.
- *Serializers.* The FileWrapper node produces a file resulting from joining the contents of the input ports with a specified extension. It can be used, for instance, to aggregate the different outputs of a workflow into a compressed file (e.g., a zip file), thereby allowing to download multiple artifacts. This node requires setting the workflow's OutputFormat to File.
- *Flow control.* The Conditional node modifies the flow control of the workflow by switching between two branches depending on the value of its predicate (a Boolean expression in EOL).
- *Miscellanea.* First, Text is a utility node that makes a text available in a workflow, without the need for creating a file. Second, the API node allows making HTTP requests to REST APIs via a URL. The node accepts an arbitrary number of input ports, whose values populate the body of the request. The node accepts methods GET, POST, PUT, and DELETE.
- *Sensemaking strategies. Model sensemaking strategies* (SMSs) are purposeful, model-based visualizations tailored for modeling environments [32]. Given their domain and level agnosticism and the model-driven foundations of Dandelion+ (cf. P1 and P2 in Sect. 4), SMSs are a good fit for understanding Dandelion+ entities: from entire Platforms, to users, DSLs, meta-models, or models. PLATFLOW offers a SensemakingStrategy node that involves three elements:

  1. The *target (meta-)model* is the artifact under study for which a visualization is produced.
  2. The *context meta-model* shapes the strategy, containing a few meta-classes describing the structure of the visual artifact (e.g., Bar, Point, or Boxplot).
  3. The *mapping* establishes a correspondence between the target (meta-)model and the context meta-model to populate the SMS.

  The node receives the target (meta-)model via input ports and applies to it a sensemaking strategy. Both the context meta-model and the mapping are described in Files: the former is written in Emfatic, whereas the latter is defined in a dedicated DSL (cf. Appendix B). Finally, SMSs admit variants, as in bar charts which can be displayed either horizontally or vertically.

[13] https://eclipse.dev/epsilon/doc/emc/#emc-drivers.

[14] https://eclipse.dev/epsilon/doc/flexmi/.

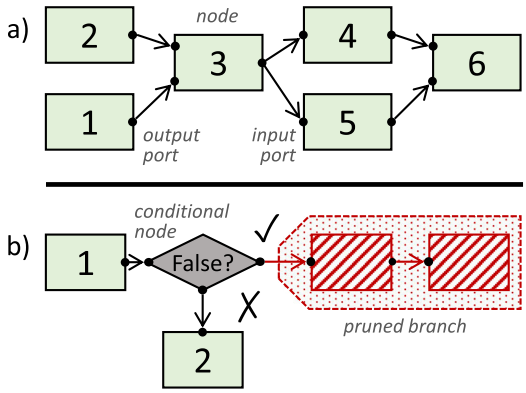


Fig. 12 Workflow execution. **a** A possible schedule of execution of the nodes of a workflow: if an output port of N_1 is connected to an input port of N_2, then N_1 must be scheduled before N_2. **b** Pruning unexecuted conditional branches: as the predicate of the conditional does not hold in this execution, the nodes pending from the true branch are discarded

6.2 PLATFLOW execution

Workflows are executed according to the dependencies imposed by the edges between the ports of the workflow nodes. In particular, to be well formed, the connections between edges and nodes must form a directed acyclic graph (DAG). The schedule of execution of nodes is then computed via a topological sort [23] of this DAG (cf. Fig. 12a). The execution engine produces the schedule incrementally to prune the unmet branches every time a Conditional node is reached (cf. Fig. 12b).

The result of the workflow is the output of the last executed node. Internally, the context scope of each node includes both one variable per input port and one variable per workflow input parameter. The name of the injected variable corresponds to that of the input port or workflow input, respectively.

6.3 Exemplary workflows

In this section, we exemplify PLATFLOW through the implementation of three workflows catered for the Mudel platform. First, Workflow #1 grades the submissions of a quiz (6.3.1); second, Workflow #2 generates a dashboard for the personnel of the platform (6.3.2); finally, Workflow #3 produces a downloadable version of the dashboard generated by Workflow #2 (6.3.3).

6.3.1 Workflow #1: quiz grading

Figure 13 presents a workflow that reports on the grades of the submissions of a quiz. The workflow is displayed in abstract syntax in the upper part of the figure, and in the concrete syntax that the stakeholders of Dandelion+ use in the bottom part.

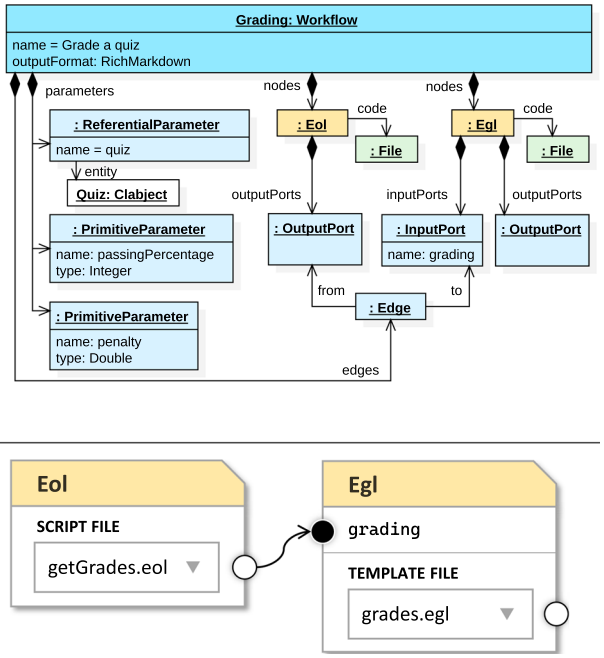


Fig. 13 Workflow #1: grading quizzes report

As the upper part of Fig. 13 shows, the interface of the workflow has three input parameters: the quiz, a passingPercentage (i.e., a number from 0 to 100), and a penalty factor (e.g., 0.25) that is applied to wrong answers. The quiz parameter points to the Quiz meta-class, allowing the professor to select the quiz to grade, while the rest are (typed) numeric PrimitiveParameters. Additionally, the output format of the workflow is set to RichMarkdown to render the generated report appropriately.

The workflow implementation logic involves two nodes. The first is an Eol node that runs an EOL script that grades the QuizSubmissions of the selected quiz (cf. Listing 1). In particular, it tallies the number of right, wrong, and unanswered questions per submission. By accounting for the penalty factor (cf. line 13), the script also determines their grades. Note that the script can use variables such as quiz (lines 2 and 3) or penalty (line 13), since input parameters are automatically injected into the execution context of every node. Moreover, the script demonstrates how EOL's dot operator (e.g., as in quiz.questions (line 2) or answer.chosenAlternative.isCorrect (line 20)) can be exploited to navigate Dandelion+ (meta-)models.

The last step generates a report using an Egl node. To do so, the node populates the template in Listing 2. This consumes a grading variable (cf. line 9) that contains the result of the previous node and is passed to the Egl node via a named input port with the same name (cf. Fig. 13). This template has the particularity of producing code in MDX[15]

[15] https://mdxjs.com.

Listing 1 Script *getGrades.eol* for grading quizzes.

```
var res = new Sequence;
var total = quiz.questions.size().asReal();
for (submission in quiz.submissions) {
  var tally = submission.tally();
  var right = tally.right;
  var wrong = tally.wrong;
  var unanswered = total - right - wrong;
  res.add(Tuple{
    "student" = submission.owner,
    "right" = right,
    "wrong" = wrong,
    "unanswered" = unanswered,
    "score" = 100.0*(right-wrong*penalty)/
total });
}
return res;
operation QuizSubmission tally() : Tuple {
  var right = self.answers.select(
    a|a.chosenAlternative.isCorrect). size();
  var wrong = self.answers.size() - right;
  return Tuple{"right"=right, "wrong"=wrong};
}
```

(an XML-like embedded language) to render special widgets in the output. This approach permits enriching an otherwise static result (i.e., just Markdown) with interactive widgets, including tables, buttons, or tabs.

Listing 2 Template *grades.egl* for grading quizzes.

```
# Grades of quiz [%=quiz.name%]
<Table>
  <Row>
    <Th>Student</Th>
    <Th>Score</Th>
    <Th>Answers</Th>
    <Th>Passed</Th>
  </Row>
  [% for (grade in grading) { %]
    <Row>
      <Cell>[%=grade.student.name%]</Cell>
      <Cell>[%=grade.score%]</Cell>
      <Cell>*OK* [%=grade.right%],
            *KO* [%=grade.wrong%],
            *NA* [%=grade.unanswered%]</Cell>
      <Cell>[%=grade.score >= passingPercentage
             ? "Passed" : "Failed"%]</Cell>
    </Row>
  [% } %]
</Table>
```

Finally, the execution of this workflow follows this order: first, the Eol node, followed by the Egl node. Since the Egl node is the last to be executed, its output becomes the output of the entire workflow, i.e., the report in Markdown. For instance, when the workflow is executed on a quiz, with a passingPercentage of 50, and a penalty of 0.5 as input parameters, it generates a report like the one in Fig. 14.

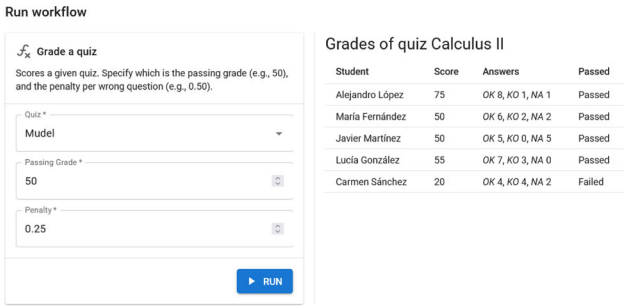


Fig. 14 Output of Workflow #1 on a Calculus II quiz, with a passing grade of 50 and a penalty of 0.25

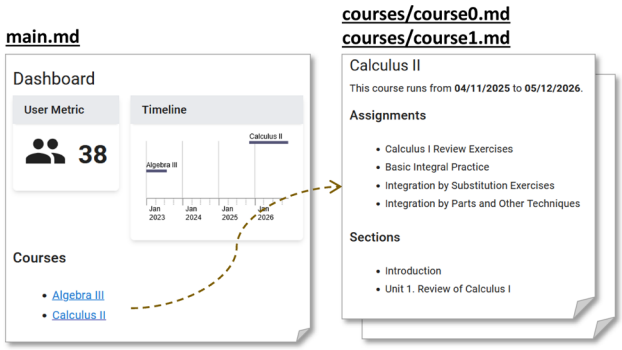


Fig. 15 Output of Workflow #2 in multiple pages

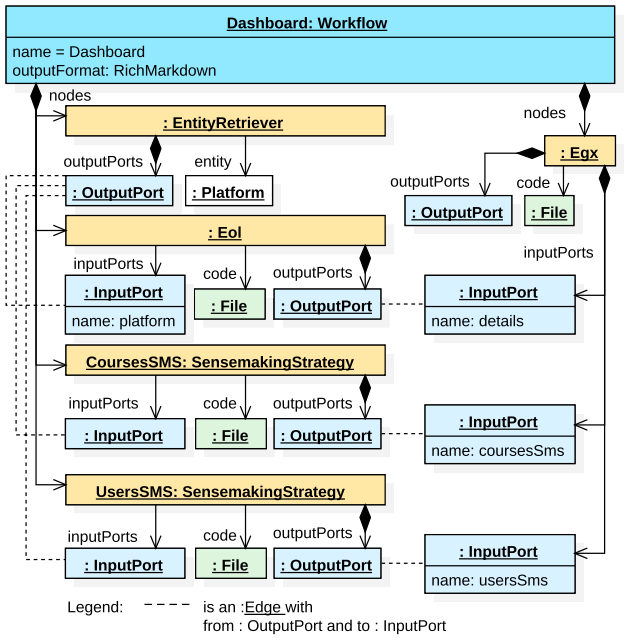
6.3.2 Workflow #2: personnel dashboard

This second workflow implements a dashboard like Fig. 15, where the personnel can get an overview of the status of the platform. Figure 16 presents the structure of the workflow.

Interface-wise, the workflow does not have any input parameter; instead, the elements it works with are directly obtained within the workflow. The output format of the workflow is set to RichMarkdown to render MDX tags, as in the previous example.

Regarding its implementation logic, the workflow execution has three phases. First, the current Platform is obtained by an EntityRetriever node. Second, the platform object is passed to an Eol node, which extracts information about the platform, and two SensemakingStrategy nodes, which produce visualizations about the current platform. Finally, these results are passed to an Egx node which, by means of a coordination script and templates, generates a series of HTML files that conform the dashboard and its subpages.

To implement a sensemaking strategy, three elements are necessary: a target (meta-)model, a context meta-model, and a mapping. For instance, the first SMS generates a timeline graph of the courses within the platform, with their start and end dates. Its target in this case is platform:Platform, and the context meta-model is in Listing 3. Context meta-models declare the concepts used by the sensemaking strategy, and each strategy defines its own meta-model [32]. The timeline



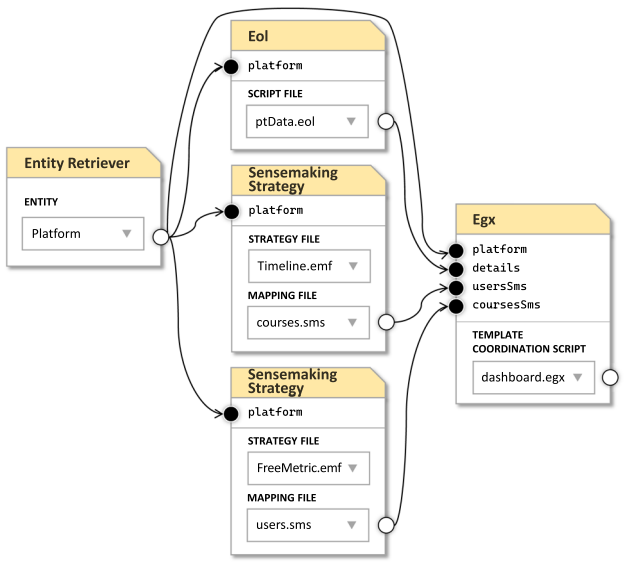


Fig. 16 Workflow #2: generating a dashboard for Mudel personnel

context meta-model only contains one meta-class, Period, that must be instantiated as many times as periods displayed in the timeline. Finally, the mapping is in Listing 4. In this case, it only contains one mapping rule (lines 1–6) that creates a period per course. Note that "this" on line 1 refers to the target (meta-)model, i.e., the current platform, and lines 3–5 map attributes of Course (i.e., name, startDate, endDate) to attributes of Period. Additionally, lines 8–10 define properties of the visualization, namely its title. Using all this information, the node produces an image that can be embedded in any HTML page.

Listing 3 Meta-model *timeline.emf*.

```
1 package timeline;
2
3 class Period {
4   attr String label;
5   attr Date start;
6   attr Date end;
7 }
```

Listing 4 Mapping *courses.sms*.

```
1  map "this.models.first()!Course.all"
2  to "Period" {
3    label <- name
4    start <- startDate
5    end <- endDate
6  }
7
8  properties {
9    title: "Courses timeline"
10 }
```

Regarding the Egx node, it generates several HTML files based on a series of templates (cf. Listing 5). In particular, it generates a main.md file, and as many Markdown reports as courses on the platform (e.g., courses/course0.md, courses/course1.md, etc.). In order to be representation-agnostic, Egx nodes always return a Java/Epsilon Map that maps the URIs of the generated files to their contents (see Listing 6). The workflow precisely returns this map.

Listing 5 Coordination script *dashboard.egx*.

```
1  pre {
2    var courses =
3        platform.models.first() !Course.all;
4  }
5
6  rule Main {
7    parameters: Map {
8      "details"=details,
9      "usersSms"=usersSms,
10     "coursesSms"=coursesSms
11   }
12   template: "dashboard/main.egl"
13   target: "main.md"
14 }
15
16 rule Subpages transform course in: courses {
17   parameters: Map { "course" = course }
18   template: "dashboard/course.egl"
19   target: "courses/course"
20           + courses.indexOf(course)
21           + ".md"
22 }
```

Listing 6 Example of a map returned by an Egx node.

```
{ "main.md": "# Dashboard ...",
  "courses/course0.md": "# Calculus II ...",
  "courses/course1.md": "# Algebra III ...", ...
}
```

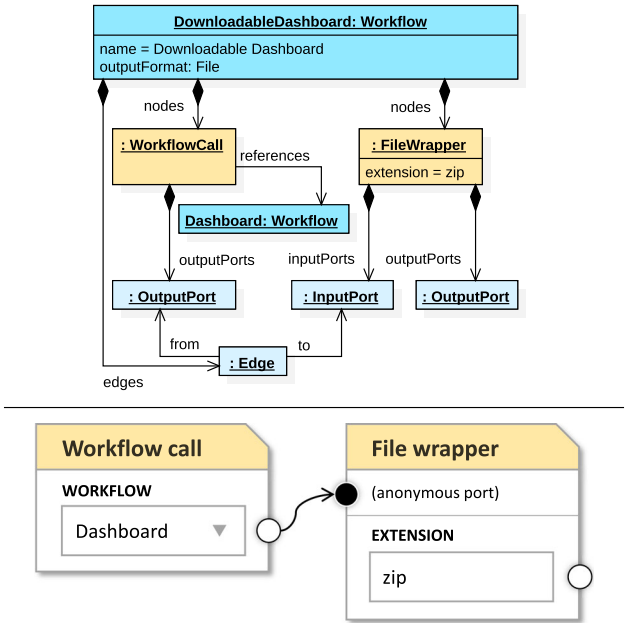


Fig. 17 Workflow #3: generating a downloadable dashboard for Mudel personnel

6.3.3 Workflow #3: downloadable personnel dashboard

Low-code platforms sometimes need to support downloading artifacts instead of just displaying them. For this reason, this last example workflow shows how to download the files produced by a workflow. Figure 17 presents its structure.

The workflow replicates the functionality of Workflow #2, with the difference that the generated report is downloaded as a compressed file containing a static webpage with its contents. To achieve this, the workflow uses two nodes. First, the WorkflowCall node allows reusing logic defined in other workflows by calling them directly. In this case, the Dashboard workflow is referenced (cf. Fig. 16), and no information is passed as input, since the called workflow does not have input parameters. Upon execution of this new DownloadableDashboard workflow, Dashboard is called and its result is passed to a FileWrapper node that bundles the information passed to it into a File. In most cases, the specified extension will leave the file as is (e.g., json, yaml, or xml). However, the zip extension is treated especially, as it expects a Map object as produced by an Egx node (cf. Listing 6) and generates a compressed file with the contents of the map.

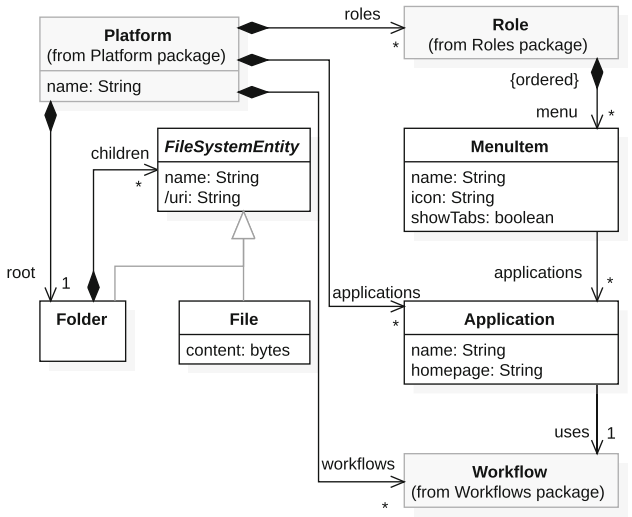


Fig. 18 The applications package

Finally, the output format of the workflow is set to File to display a download button correctly.

6.4 Application deployment meta-model

So far, we have explained how PLATFLOW allows defining workflows. While all workflows can be executed standalone, some of them can be deployed as running parts of the Dandelion+ interface. We achieve this with *applications*. Figure 18 models applications, together with the support for file systems.

Regarding file systems, each platform has one, and they comprise Folders, which may be nested, and Files. This way, files with any content may be stored in an ordered fashion, from scripts to documents and images. Both Files and Folders are identifiable via a URI in code or in the workflow editor.

As per workflows, some of them can be deployed as Applications. In this context, an application is an indivisible collection of pages that fulfill a purpose. For this reason, only workflows that produce collections of pages are eligible for deployment. In PLATFLOW, the node that generates such pages is Egx[16]: it produces a map from strings to strings, which can be interpreted as a mapping URL $\mapsto$ ⟨page contents⟩, i.e., a collection of pages. For instance, whenever Workflow #2 (6.3.2) is executed, it produces a map as the one in Listing 6. Therefore, the only required information to deploy the workflow is setting its entry point to main.md via Application.homepage. Please note that pages of any format can be served: from HTML, to Markdown or any other format (e.g., *.txt).

[16] To be more precise, only workflows that return a map with strings as keys and values can be deployed as applications.

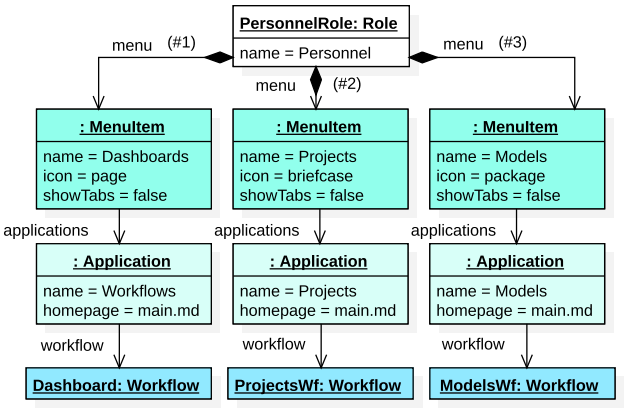


Fig. 19 Example of workflow deployment in applications

Figure 19 shows how to model the application side of the PersonnelRole. The role has a menu consisting of three items: *Dashboard*, *Projects*, and *Models*. Each of them is tied to a single application that deploys a workflow. In particular, the former deploys Workflow #2 (6.3.2), whereas the latter provide pages for the projects (meta-models) and models of the platform. These last workflows are created by default.

7 Architecture and tool support

This section presents the architecture 7.1 and tool support 7.2 of Dandelion+.

7.1 Architecture

Figure 20 presents the architecture of Dandelion+, which is divided into a *frontend* and a *backend* component.

The frontend exposes a web-based application available for all stakeholders (i.e., LCDP engineers, citizen developers, and end users). Technologically, the frontend makes use of React [14] to build the user interface together with MUI,[17] a React component library. The graph-based editors are implemented using React Flow,[18] and the *Files* page makes use of the Ace Editor.[19] Finally, the frontend communicates with the backend via a REST API, allowing the adoption of alternative frontends in the future.

As for the backend, it is the heart of the tool, as it manages platforms and all their elements (e.g., (meta-)models, workflows, or roles), namely a PLATFLOW service, to define and execute workflows; a layout service by the Eclipse Layout Kernel (ELK) [55] to arrange elements automatically in the diagrams; a Sensemaking Strategies service

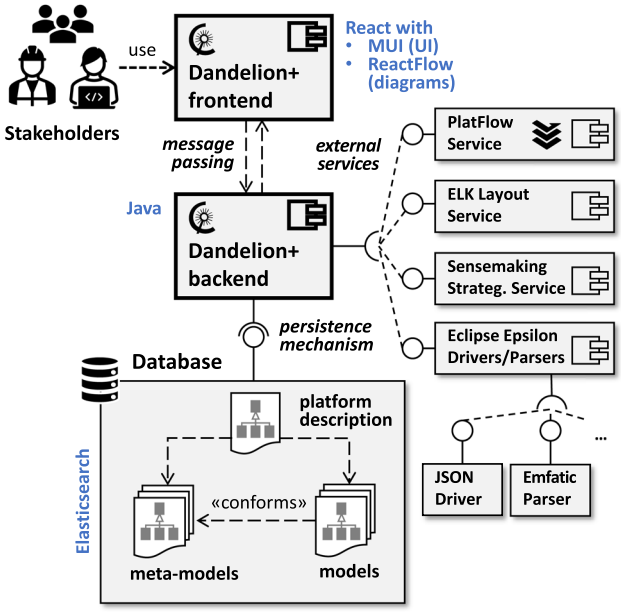


Fig. 20 Architecture of Dandelion+

to process the sensemaking strategies defined in the workflows [32] (cf. node SensemakingStrategy in 6.1.3); and Epsilon tooling, including drivers and parsers, to interact with different technologies, such as JSON, Emfatic, or FLEXMI. The backend is implemented in Java, as PLATFLOW AND THE STRUCTURE META- MODEL ARE IMPLEMENTED ATOP EPSILON, WHICH IS JAVA- BASED.

Finally, the persistence layer is decoupled from the backend thanks to the EMC layer by Epsilon, thereby allowing the LCDP engineers to choose their most suitable persistence technology. Our current implementation leverages Elasticsearch [13], as we demonstrated its scalability and speed for MDE-based low-code applications in [31], where all elements related to platforms are persisted.

7.2 Tool support

This section showcases the functionality of Dandelion+, which is offered as a website. To do so, it illustrates how to develop the Mudel platform (cf. Sect. 3) from scratch, following the process presented in Fig. 3. In particular, this section explains how Dandelion+ manages platforms (7.2.1), how to develop the structure (7.2.2) and behavior (7.2.3) of a platform.

7.2.1 Managing platforms

The entry point of the tool is the *top-level homepage*, which lists all available platforms and permits logging into them (cf. Fig. 21).

[17] https://mui.com.

[18] https://reactflow.dev.

[19] https://ace.c9.io.

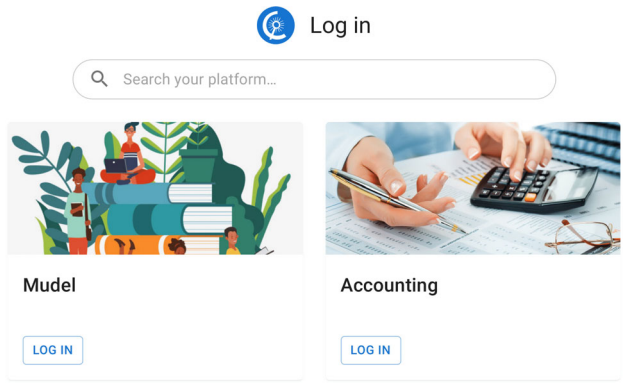


Fig. 21 Top-level homepage

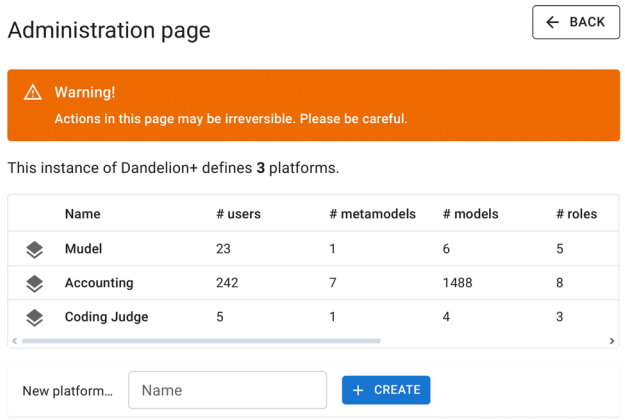


Fig. 22 Administration page

From there, the *administration page* (Fig. 22) is also available, allowing to manage the platforms defined in the system. The page provides metrics about them, such as the number of users, meta-models, or models. Its access is protected by a password set during the installation of Dandelion+.

Once a platform is created from this page, the system automatically populates multiple entities for its correct functioning, namely an administrator Role; a User with that role, allowing the LCDP engineer to log in and start developing the platform; workflows for predefined pages (e.g., Projects, Models, Workflows, etc.); and applications for these workflows.

7.2.2 Developing the structure of a platform

To develop the structure of a platform, users need to log into it. The first page users encounter is the platform *Homepage* (cf. Fig. 23). All pages have a header and a navigation menu, which links to the applications defined in the platform. The ones included by default are: the *homepage*, *projects*, *models* (i.e., meta-models and models), *workflows*, *files*, *applications*, a *chatbot*, and *platform settings*.

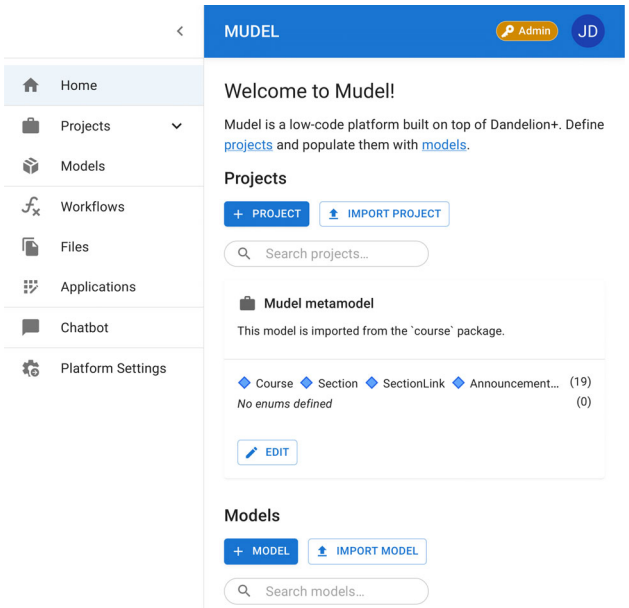


Fig. 23 Homepage of the Mudel platform

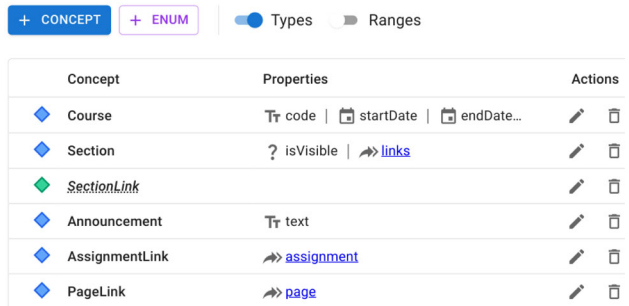


Fig. 24 Tabular meta-model editor

The *Projects* page allows managing meta-models. Dandelion+ features two flavors for meta-model edition: tabular and graphical. Both allow manipulating concrete and abstract classes, enumerations, properties, literals, associations, and containments.

On the one hand, the tabular editor (cf. Fig. 24) permits hiding or showing both types and ranges (i.e., multiplicities), to better adapt the level of detail of exploration of a meta-model. Note that, although the tabular representation is appropriate when meta-models are very dense, they pose a problem regarding the graph-like nature of meta-models. In particular, acknowledging the relations between classes may be difficult. To alleviate this issue, relations are marked as clickable links which point to their referred types. For instance, when clicking links on the class Section, the page scrolls down to the class SectionLink and highlights that row momentarily.

On the other hand, the graphical editor (cf. Fig. 25) is interactive, allowing to move elements within the canvas, as well as panning and zooming. It also features automatic layout

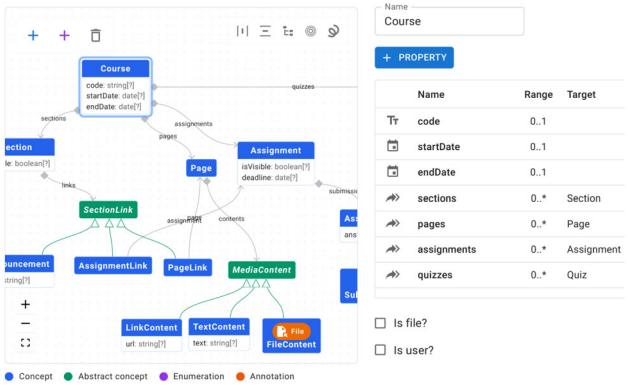


Fig. 25 Graphical meta-model editor

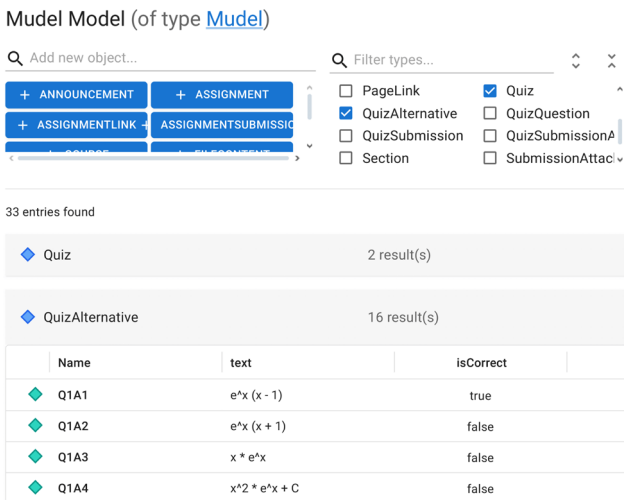


Fig. 26 Model editor

facilities out of the box, including horizontal, vertical, tree-like, circular, and force-based arrangements. Classes can be marked as abstract and decorated with User and File annotations. These are reflected in the diagram (e.g., FileContent in the figure).

Once a meta-model has been created, the user can click on the *Models* link to instantiate it. Figure 26 presents the *model editor*, where the objects are arranged tabularly and organized by types, displaying all their properties. Types can be hidden or made visible via checkboxes. The panels containing the objects of each type can also be expanded or collapsed to examine the model. Moreover, existing objects can be modified, deleted, and new ones can be created.

7.2.3 Developing the behavior of a platform

The *Workflows* page permits managing the workflows defined in the platform (cf. Fig. 27). In particular, it permits modifying them, deleting them, and creating new ones.

Figure 28 showcases the workflow editor. It allows giving workflows a name, a description, input parameters, output

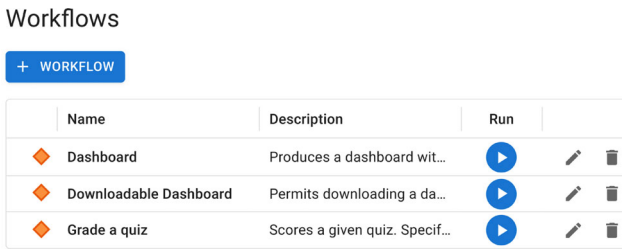


Fig. 27 Workflows page

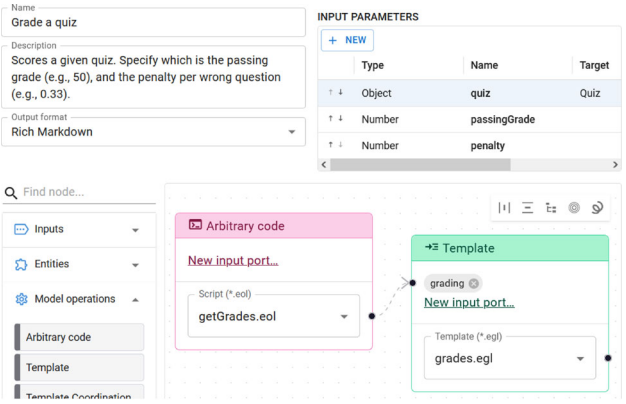


Fig. 28 Workflow editor



Fig. 29 Files page

type, and implementation. Input parameters can be typed with either primitives (e.g., string, numbers, or Booleans) or Dandelion+ types, as presented in the linguistic meta-model in Sect. 5 (e.g., model, meta-model, role, etc.). The visual editor allows drag and drop of nodes in the left, which are distributed in the categories presented in 6.1. Some nodes include search boxes that filter through the entities of the corresponding type by name for easy selection.

Workflows can be tested iteratively while *running* them from the *Workflows* page (e.g., Fig. 14). As shown, the workflow is rendered as a form where input parameters are mapped to inputs, checkboxes, or selects according to their typing.

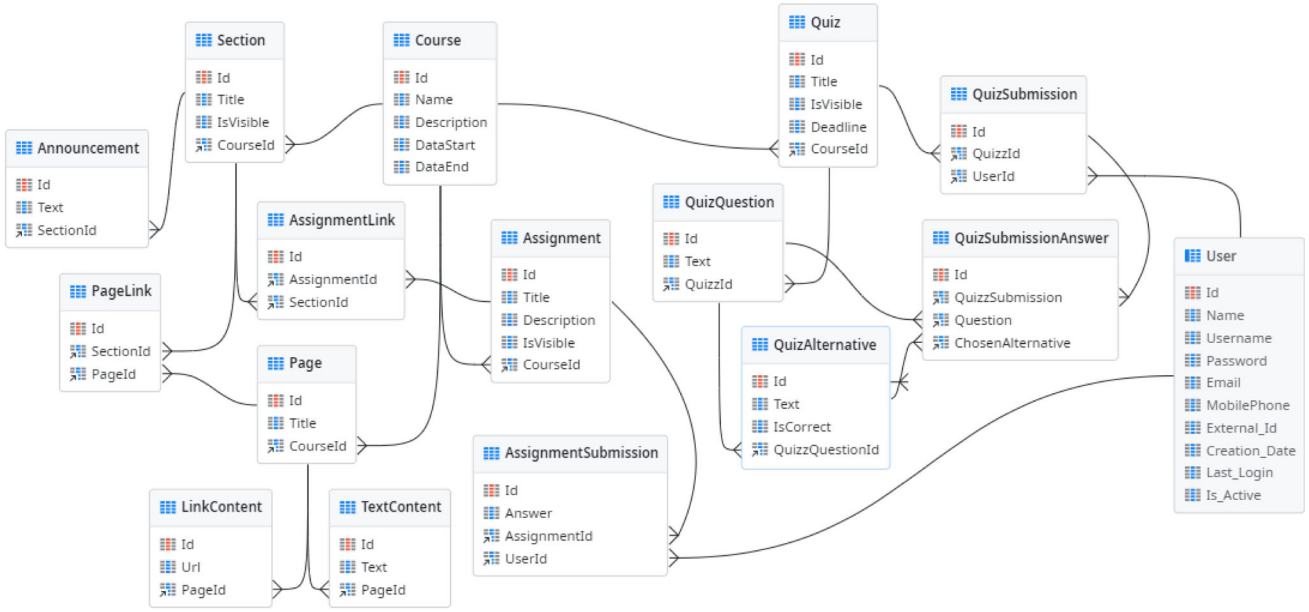


Fig. 30 Database schema of Mudel in OutSystems

The visual appearance of the result of an execution depends on the output type.

The *Files* page allows manipulating the files used in the workflows, as shown in Fig. 29. In particular, the page lists all files and folders, and the right-hand-side editor allows editing the files as well as saving them.

Dandelion+ integrates with LowcoBot, a conversational assistant for low-code platforms [33]. In particular, a model of Dandelion+ is implemented in it, covering the graphical, programmatic, and ontological sides of the tool. The result is a chatbot able to interact with the platform and help the user through the website (e.g., *Where can I define a new role?*). A screenshot of the integration is available in Figure 6 in [33].

Finally, the *Platform Settings* page permits manipulating the roles, their menus, and managing users.

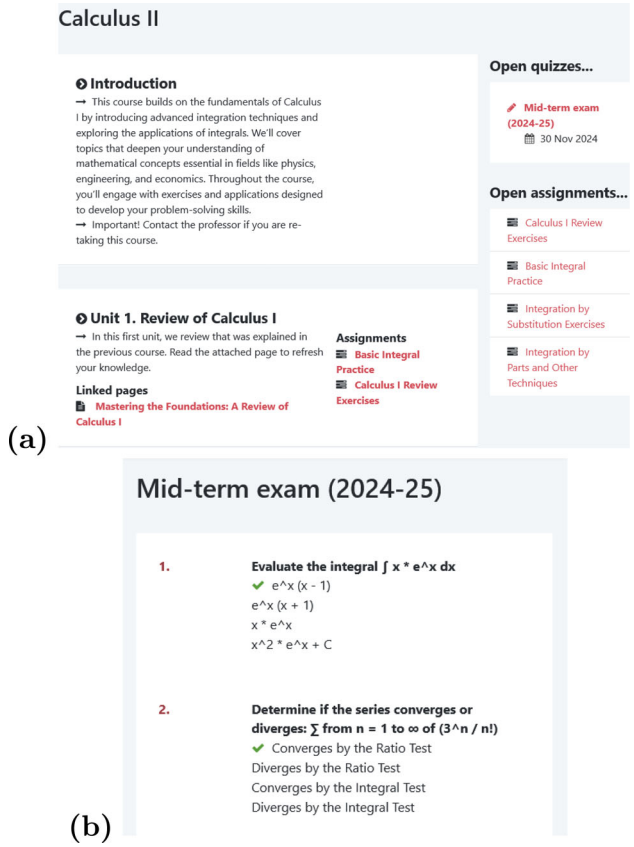


Fig. 31 Pages defined in OutSystems: **a** course detail (students, personnel); **b** quiz detail (professors)

8 Evaluation

This section evaluates the competitiveness and suitability of Dandelion+ through these research questions:

RQ1. How does Dandelion+ compare to other low-code platforms to build applications that require code generation?

RQ2. How can Dandelion+ improve UGROUND's industrial process to create low-code solutions?

In the following, Sect. 8.1 explores RQ1, and Sect. 8.2 explores RQ2. Then, Sect. 8.3 discusses the threats to validity of the evaluation. The artifacts and code used in this evalua-

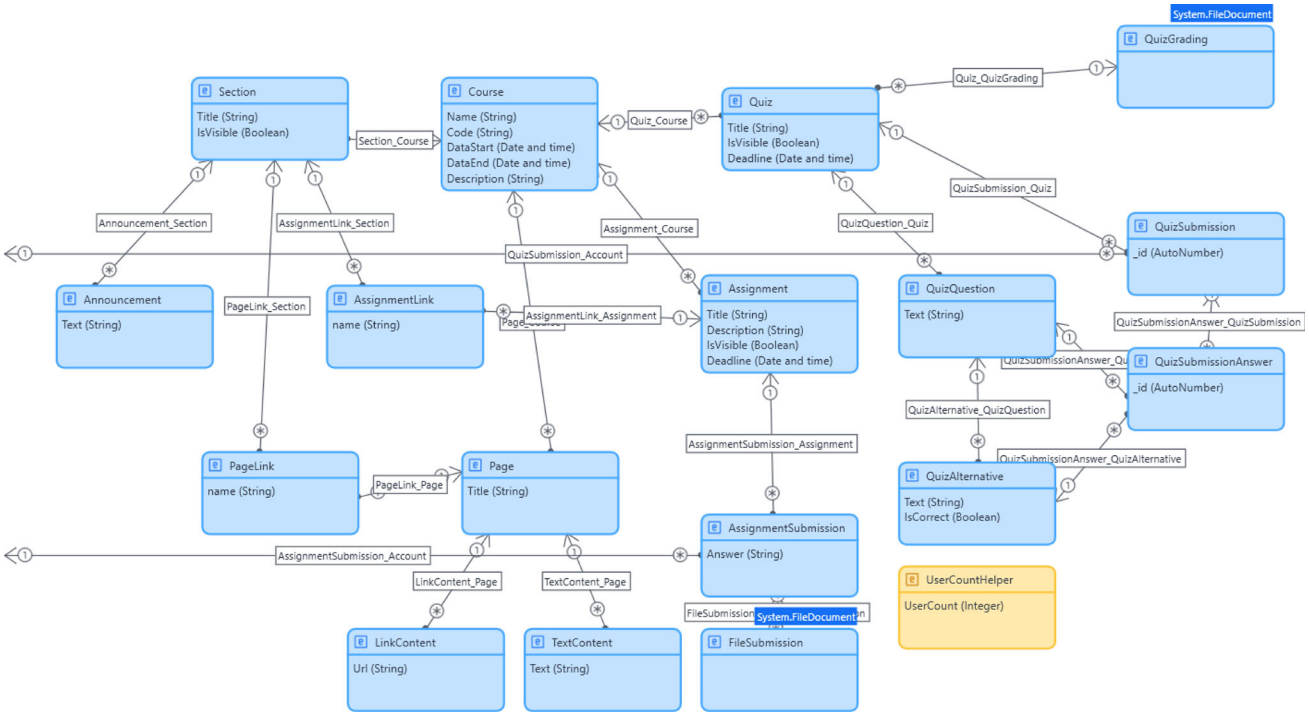


Fig. 32 Domain model of Mudel in Mendix

tion, as well as a catalog of screenshots of the execution of these artifacts, are available at Dandelion+'s website.[20]

8.1 RQ1: Comparison with other low-code platforms

In this research question, we evaluate the use of Dandelion+ to build applications that require code generation. For this, we compare Dandelion+ with state-of-the-art low-code platforms, as reviewed in Sect. 2.

Specifically, we model Mudel (the running example from Sect. 3) using Dandelion+ and compare our solution with similar implementations in OutSystems [40], Mendix [34], and BESSER [1]. We selected OutSystems and Mendix because they are full-fledged low-code platforms widely used in industry. As for BESSER, we included it as one of the few academic low-code platforms, sharing a model-driven foundation with Dandelion+. Overall, our goal is to cover a broad spectrum of low-code platform features in our evaluation.

8.1.1 OutSystems

We have implemented the Mudel domain using Service Studio, OutSystems' desktop IDE.

Regarding the structure of the platform, its domain is modeled in the database schema shown in Fig. 30. The schema mostly covers the Mudel domain model, with some limitations. First, as OutSystems relies on a relational persistence schema, inheritance relationships cannot be expressed natively. To circumvent this limitation, object-relational mapping strategies, such as single, class, or concrete table inheritance patterns [15] can be applied. Still, the semantics of the original model is not preserved. In our case, we apply the single table inheritance pattern to SectionLink and MediaContent and their respective subclasses. Second, concepts FileContent and AssignmentAttachment (cf. Fig. 7) cannot be represented, as web applications in OutSystems do not support file uploads. Finally, object ownership can be modeled by importing User (an entity defined at a system level) and associating it with the ownable entities.

Regarding roles, we have defined all the proposed ones in Table 2 (i.e., Professor, TA, Student, and Personnel). Concerning pages, we have implemented a homepage, a courses list, a course detail, as well as creation or delete screens for all the entities. Some pages had to be duplicated and adapted to cater for the roles in the platform. Figure 31 shows some screenshots of the application.

Regarding behavior, we could not implement any of the workflows, as OutSystems server actions (the ones that manage data) can only execute SQL queries to perform complex operations. While SQL is a powerful language (and potentially Turing-complete), this approach is unfeasible to implement the behavior of the platform, including scoring quizzes.

[20] https://miso.es/tools/Dandelion.html.

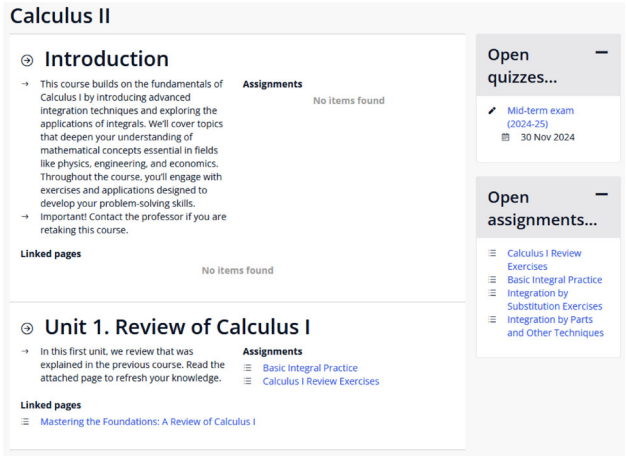


Fig. 33 Course view in Mendix

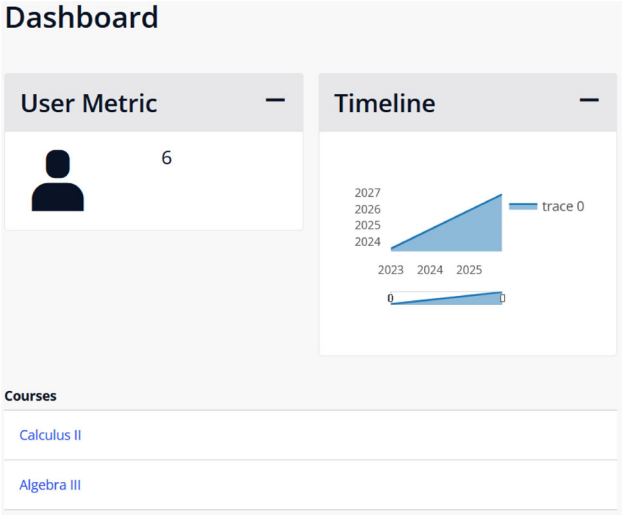


Fig. 34 Dashboard view in Mendix

8.1.2 Mendix

For Mendix, we have used Mendix Studio. Regarding the structure of the platform, the entities are captured in the *domain model* shown in Fig. 32. The schema covers the entirety of the Mudel domain. First, inheritance is supported for meta-classes like SectionLink and MediaContent through *generalization*. Second, FileSubmissions can be represented using the mechanism of generalization over the Mendix's system-defined concept of FileDocument. Third, some additional entities are required as temporary types, such as QuizGrading and UserCountHelper.

Regarding roles, Mendix has a similar approach to Dandelion+: all can be defined (from Professor to Personnel), each having a different menu and scope of executable workflows, as well as restricted pages.

Regarding pages, we have implemented a homepage, a course detail, a dashboard, and details for all the entities of the platform. Mendix automatically generates them with a "Generate overview pages..." option. In particular, in Fig. 33, we could not replicate conditional rendering for empty-related assignments nor related pages. In Fig. 34, we could replicate the user metric and the course enumeration and detailing, but not the course timeline, as it is not an available graph.

Regarding behavior, we could implement a PDF generation of Workflow 3, although we could not populate it with the necessary data. While Mendix has *microworkflows* to process data, facilities to implement business logic are very reduced.

8.1.3 BESSER

BESSER is a low-code tool that aims to "*model, generate, personalize and deploy smart and complex software systems*."[21] Its approach starts with the user providing a meta-model expressed in B-UML (BESSER's Universal Modeling Language), from which code generators are executed to produce code targeting the desired technologies. Currently, meta-models can be imported through PlantUML, custom Python codes, or images of class diagrams. Regarding code generators, the tool supports multiple generators targeting, among others, Django, REST APIs with Python's FastAPI, database schemas, or Flutter applications.

In our case, we have expressed the meta-model using PlantUML, as its semantics coincides with that of the class diagram employed in the running example. Regarding the generated artifacts, we have used Django, as it produces web applications and automatically exposes an administration section (i.e., Django's admin site) that allows managing the application's data at runtime in a low-code fashion.

Regarding the meta-model, we have replicated the meta-model shown in Fig. 2 in PlantUML syntax with some modifications. In particular, we have omitted the User class and the Role enumeration, as these concepts are handled natively by Django. Additionally, we have had to remove the classes that manage files (FileContent and AssignmentSubmission), as BESSER's Django generator presently does not handle file fields,[22] although Django supports them natively.[23]

Regarding roles, Django manages them through *groups*. We have defined groups *Student*, *Professor*, *TA*, and *Personnel* (cf. Fig. 35).

[21] https://besser.readthedocs.io/en/latest/index.html.

[22] https://github.com/BESSER-PEARL/BESSER/blob/master/besser/generators/django/templates/django_fields.py.j2.

[23] https://docs.djangoproject.com/en/5.1/ref/models/fields/#filefield.

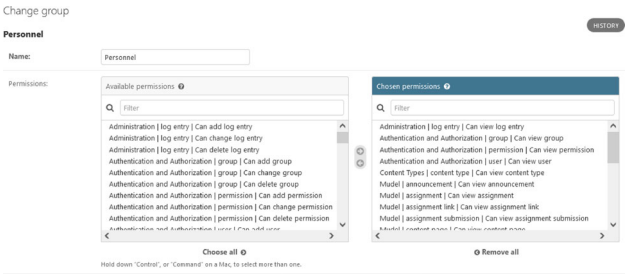


Fig. 35 Group (role) 'Personnel' defined in Django

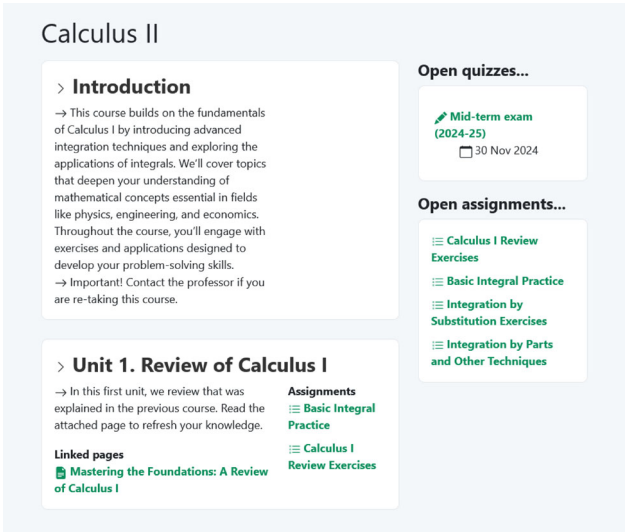


Fig. 36 Page defined in Django out of a BESSER model: course detail (students, personnel)

Regarding pages, these are defined using *views*,[24] which typically load models from the database and render them using templates employing the Django template language,[25] which is similar to Jinja and placeholder-based as well. Figure 36 shows one of the generated pages out of a BESSER model.

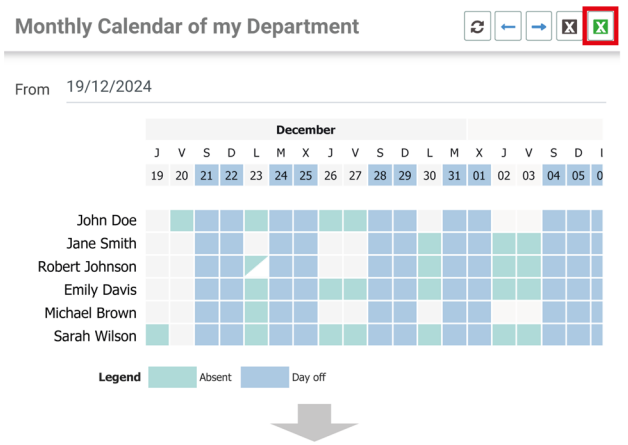
When it comes to behavior, BESSER currently seems to only generate static information from the input meta-model. Any interactivity must be programmed using Django ORM in Python, defeating the purpose of a low-code-contained solution.

8.1.4 Discussion

Table 4 summarizes the main features of the studied low-code platforms for replicating the Mudel domain. We distinguish three main feature categories when it comes to this evaluation: domain model construction, user interface description, and behavior specification.

[24] https://docs.djangoproject.com/en/5.1/topics/http/views.

[25] https://docs.djangoproject.com/en/5.1/ref/templates/language.



	A	B	C	D	E	F	G	H	I	J	K	L	M
1	**Holiday Report 2024**												
2		JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC
3	John Doe	3	4	4	1	0	0,5	0	9	0	1	0	3
4	Jane Smith	4	3	4	0	0	0	5	1	3	0,5	0	1,5
5	Robert Johnson	0,5	0,5	2	3	0	2,5	3	4	0	4	1,5	1,5
6	Emily Davis	0	0	3	0	0	0	0	16	0	0	0	4
7	Michael Brown	4	2	0	0	0	0	3	9	0	0	0	1
8	Sarah Wilson	1,5	1,5	4	1	0	0	5	5	1	0	0	5

Fig. 37 Monthly calendar report: within UGROUND's platform and exported as an Excel spreadsheet

Regarding domain model construction, commercial LCDPs predominantly use a database schema or an equivalent approach, whereas BESSER and Dandelion+ adopt a model-driven methodology. The level of support for modeling features varies across platforms. Most LCDPs support inheritance and object ownership through different mechanisms. OutSystems achieves object ownership by reusing (i.e., importing) a predefined User table/entity at the system level. In contrast, Mendix and Dandelion+ implement ownership via "generalization" or annotations. File persistence support is more restricted, with only Mendix and Dandelion+ providing full support, while BESSER offers partial support. Finally, all platforms feature a graphical editor, facilitating the visual construction of the domain model.

To describe the user interface of the target application, OutSystems and Mendix rely on visual drag-and-drop editors, making them easier to use and amenable to non-technical users. Conversely, BESSER and Dandelion+ employ a coding approach that offers greater flexibility: BESSER specifically uses Django's HTML-based template language, while Dandelion+ leverages Epsilon's EGL and EGX to generate HTML and other formats, such as enriched Markdown, code snippets, or validation reports.

For behavior specification, all LCDPs follow a workflow-based approach. However, the expressiveness of these workflows is generally very limited. Specifically, OutSystems and Mendix fail to fully reproduce Workflows #1–#3, as their workflow constructs and SQL query capabilities are insufficient to capture the required behavior. BESSER, which

Table 4 Qualitative comparison of different low-code platforms for replicating the Mudel domain

Feature	OutSystems	Mendix	BESSER	Dandelion+
Domain model	DB schema	Object schema	Model-based	Model-based
Inheritance	○	●	●	●
Object ownership	● (reuse entity)	● (generalization)	○	● (annotation)
File persistence support	○	●	◒	●
Graphical editor	●	●	●	●
User interface	Drag-and-drop	Drag-and-drop	Template (HTML)	Template (flexible)
Behavior	Workflow-based	Workflow-based	—	Workflow-based
Model transformation	Insufficient	Limited	*(Before runtime)*	Full (MDE-based)
Custom code generation	Insufficient	Limited (PDF, HTML)	*(Before runtime)*	Full (agnostic)

Table 5 Quantitative comparison of different low-code platforms for replicating the Mudel domain

Feature	OutSys	Mendix	BESSER	Dand+
Domain model				
# meta-classes	16	18	17	18
# annotations	0	3	0	2
# inheritances	0	0	5	6
model LoC	—	—	92	—
User interface				
# visual nodes	18	15	—	—
inferface LoC	—	—	307	49
Behavior				
WF #1 OK?	○	○	—	●
WF #2 OK?	○	◒	—	●
WF #3 OK?	○	◒	—	●
# wf nodes	0	7	—	9
# wf edges	0	5	—	9
scripts LoC	—	—	21	69

generates greenfield applications from a modeled specification, applies model transformation and custom code generation at design time rather than runtime. However, in a strict sense, this does not fully align with the definition of behavior specification, as behavior should be enacted at runtime. In contrast, Dandelion+ successfully generates all required artifacts using an MDE approach, supporting both platform-agnostic model transformations and custom code generation.

Our qualitative observations are supported quantitatively in Table 5, which collects relevant metrics from the different platforms used to replicate the Mudel domain. First, for domain model construction, we observe that the number of meta-classes is very similar across platforms, showing that most of the domain could be captured in all of them. The minor differences stem from particular treatments of inheritance, object ownership, and file persistence (e.g., through annotations). We also note that BESSER is the only platform that expresses the domain model in a textual format (i.e., PlantUML). Second, regarding user interface, OutSystems and Mendix employ a similar number of visual nodes, as the expressivity of their UI languages is very similar. In contrast, BESSER and Dandelion+ make use of code. We note that, since Dandelion+ templates employ PLATFLOW's enriched Markdown, which is more expressive than Django's HTML-based template language, it requires less lines of code. Finally, regarding behavior implementation, we observe that OutSystems does not allow to implement any of the three workflows, Mendix is very limited, and Dandelion+ manages to implement all of them. The total number of nodes and edges in Mendix and Dandelion+ is similar, although Dandelion+ implements the three workflows. As a last remark, BESSER and Dandelion+ require some scripts to enable behavior within their applications: in BESSER, for implementing the views (Django is a model-view-controller framework) and in Dandelion+ for the implementations tied to the workflows' nodes.

In conclusion, we answer RQ1 by stating that Dandelion+ stands out over other LCDPs when building applications that require code generation. For structure, Dandelion+ provides key features for constructing complex domain models, such as inheritance, files, and object ownership. For behavior, Dandelion+ offers a more expressive workflow-based behavior specification toolkit, improving those of competing LCDPs that lack the necessary expressiveness for this use case.

8.2 RQ2: Improving industrial UGROUND process

In this research question, we evaluate the advantages of Dandelion+ to realize low-code solutions in an industrial setting. In particular, we compare UGROUND's organizational digital twin development environment with Dandelion+ workflows, in order to assess the benefits of the latter.

UGROUND [60] is a Spanish software firm that engineers solutions for different business sectors, such as banking, insurance, or human resources. To do so, UGROUND follows a methodology that mixes model-driven development and digital twins. In particular, the company builds low-code platforms that work as organizational digital twins atop ROSE [11], a patented ontology-driven modeling engine.

8.2.1 Developing behavior in UGROUND

The development process of features in UGROUND mixes traditional programming with a model-driven approach, namely the company uses JCORE, a clone of JavaScript, to develop platform features, and ROSE for platform structure.

Figure 37 presents the monthly calendar report developed using UGROUND's platform[26]: it tabulates the number of off and absent days per employee. We will focus on the last action of the report, highlighted in red. When clicked, this action generates an Excel spreadsheet with the number of taken holiday days per month per employee, as seen in Fig. 37. These buttons are called *actions* in UGROUND, and their creation or modification entails the next steps:

(1) Pinpoint the page type and the action code. In order to serve pages like the one in Fig. 37, UGROUND platforms do not execute JCORE code directly. Instead, an intermediate representation layer is generated automatically in a domain-specific XML format (cf. Listing 7). This markup language describes the visual components of the page, from which the frontend can render the appropriate HTML DOM using a templating system. In order to examine this intermediate code, the developer must open the log tracer of the platform (a desktop application). Three pieces of information are important here. First, the window is of type TSVDIR (line 2). This means that the implementation of this page/window is located at a tsvdir.cor file. Second, the page defines a control (i.e., a field) called START_DATE with a "From" label of type date (line 5). This control is used to set the start date of the holiday report, which is a raw HTML dump in line 13. Finally, the action of interest is located in line 11, and its procedure name bridges the visual component and its execution.

(2) Modify/create the action in the class file. Listing 8 shows the class tsvdir.cor that we have identified in the previous step. Classes in JCORE are divided into three sections: data (line 2), page structure (lines 4–13), and functions (lines 17–26). Data are the fields of the current page. Page structure is described by constants and interfaces. In particular, constants define the placement of the visual elements (e.g., fields and controls). In the example, lines 4–7 are the code definition of Listing 7. Constants consume interfaces by name. Note that our action (line 13) is consumed by name in line 4, and the last parameter in line 13 points to the function to be executed.

Listing 7 Generated XML artifact as intermediate concrete syntax

```
1  <?xml version="1.0"?>
2  <Window type="TSVDIR">
3   <Title>Monthly Calendar of my
        Department</Title>
4   <Page>
5    <Control type="date" title="From"
         target="START_DATE"> 19/12/2024</Control>
6    <Control type="ToolBar" for="TSVDIR">
7     <bO procName="REFRESH" alt="Refresh" />
8     <bO procName="PREVVACATIONPERIOD" alt="30
          previous days" />
9     <bO procName="NEXTVACATIONPERIOD" alt="30
          next days"/>
10    <bO procName="EXPORTTOEXCEL" alt="Export to
          Excel"/>
11    <bO procName="EXPORTTOEXCELANNUAL"
          alt="Export to Excel Holiday Report" />
12   </Control>
13   <Control type="html"> ... </Control>
14  </Page>
15 </Window>
```

In our case, the function (lines 17–24) creates an XLS file out of a generated HTML report with the contents of the spreadsheet. This approach is followed as many JCORE libraries make an extensive use of HTML as a source format from which to convert to other formats. Finally, the spreadsheet file is returned. Should we want to change the structure of the generated Excel file (line 21), then the function GetYear from class CalendarAux (located in calendaraux.cor) should be modified. As a reference, this file has 98 lines of imperative code. When a new action is added, the constant in line 4 has to be altered, and both an interface and a function should be created and linked appropriately by name.

(3) Push the changes and synchronize them in ROSE. Once the changes are persisted in the version control, the developer accesses the ROSE platform in the browser and synchronizes the changes. By clicking on the 'synchronize' button, ROSE diffs the changes and updates its model about the interfaces and functions of the platform. If there are conflicts, they will be notified. At this stage, the developer may want to document the created or modified page structure elements.

(4) Regenerate the code. In order to enforce conformity between the model and the JCORE code, the code must be regenerated after committing changes through ROSE. In this step, static analysis is performed to raise issues, and a linter is applied.

(5) Deploy the changes. Once the code is regenerated, another developer from the platform-as-a-service (PAAS) department accesses the (pre)production servers and stops

[26] Names have been removed to preserve privacy.

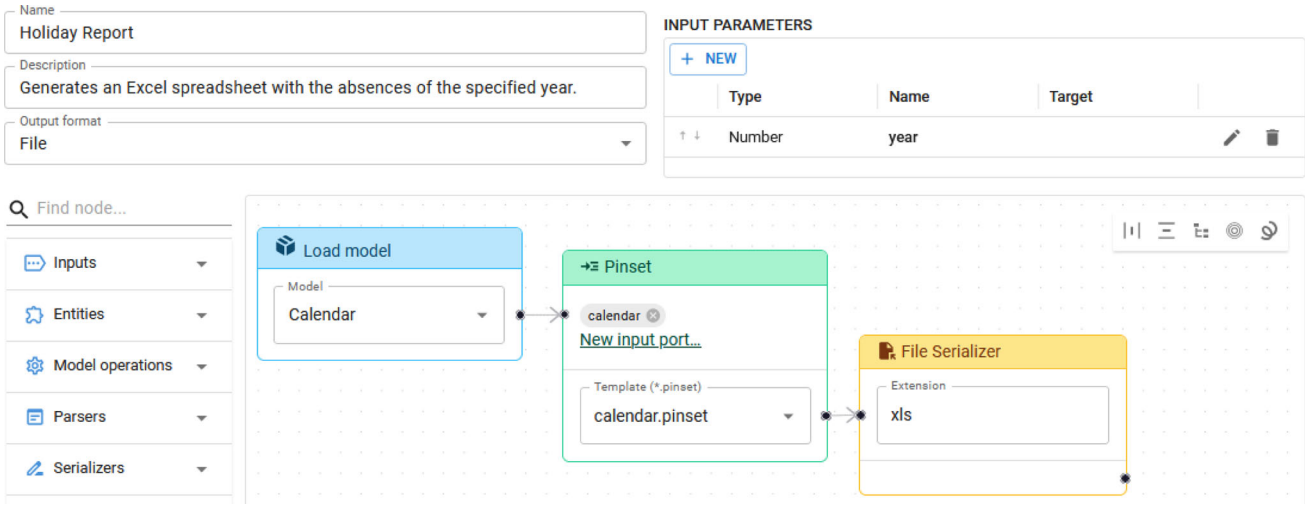


Fig. 38 Holiday report workflow in Dandelion+

the running service. Next, he/she gets the last version of the code and restarts the service. This completes the feature development cycle.

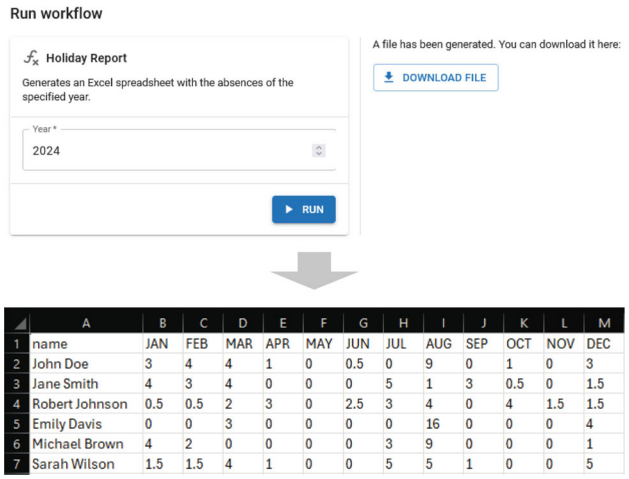
8.2.2 Dandelion+ approach

In Dandelion+, we make use of workflows to replicate the same action.

Figure 38 presents the holiday report workflow. The workflow has a year as an input parameter; it loads a calendar model, applies a Pinset node, and returns a file with the generated tabular content as an XLS file.

Listing 9 shows the script of the Pinset node. This node consumes the provided model and generates a tabular structure with months as columns and employees as rows. Each cell counts the number of absences (counting only half point per half-day absence).

Figure 39 shows the Excel file that is downloaded when the workflow is executed for the year 2024.



	A	B	C	D	E	F	G	H	I	J	K	L	M
1	name	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC
2	John Doe	3	4	4	1	0	0.5	0	9	0	1	0	3
3	Jane Smith	4	3	4	0	0	0	5	1	3	0.5	0	1.5
4	Robert Johnson	0.5	0.5	2	3	0	2.5	3	4	0	4	1.5	1.5
5	Emily Davis	0	0	3	0	0	0	0	16	0	0	0	4
6	Michael Brown	4	2	0	0	0	0	3	9	0	0	0	1
7	Sarah Wilson	1.5	1.5	4	1	0	0	5	5	1	0	0	5

Fig. 39 Excel file generated by the holiday report workflow in Dandelion+

8.2.3 Discussion

We observe differences and similarities between UGROUND's and Dandelion+'s approaches.

We identify the following differences. First, in UGROUND, deploying changes implies regenerating code manually, stopping the platform, and restarting the system. In contrast, in Dandelion+, both the structure and behavior of workflows can be changed at runtime, allowing a rapid and dynamic development. Second, the approaches differ in platform containment. In Dandelion+, a web browser is the only necessary development tool, while in UGROUND, an IDE and a dedicated log tracer are required in a desktop environment. Third, Dandelion+ fully relies on model-driven principles, offering a set of model management languages of the Epsilon family. This allows using the most appropriate language for the task at hand. In the example, we used Pinset's declarative rules to map a model into a tabular structure, instead of relying on a lower-level programming language. Finally, the approaches manage format conversions differently. Like UGROUND, Dandelion+ can generate Excel files from models. However, when processing Excel files to produce other artifacts (i.e., the reversed data processing), Dandelion+ uses dedicated drivers (e.g., for JSON files) to yield operable models. This allows for targeted management through model-driven workflow nodes that use tailored DSLs.

Regarding their similarities, both approaches share a foundation in model-driven principles: the structure of their applications and actions (workflows) is represented and persisted using models. However, Dandelion+ goes a step

Listing 8 File tsvdir.cor in JCORE

```
class TSVDir extends Window {
 data START_DATE

 constant PAGE_SHOW_DEPT_MONTH_CAL =
  "#date,From,START_DATE;" +
  "ToolBar,REFRESH,PREVVACATIONPERIOD,
      NEXTVACATIONPERIOD, EXPORTOEXCEL,
      EXPORTTOEXCELANNUAL;" +
  "#html,HTML_STR;"

 interface REFRESH("Refresh","m_repeat",
     SetUsrDeptMonthCalHTMLBtn)
 interface PREVVACATIONPERIOD("30 previous
     days", SetPrevVacationPeriod)
 interface NEXTVACATIONPERIOD("30 next
     days", SetNextVacationPeriod)
 interface EXPORTOEXCEL("Export to Excel",
     DoExportToExcel)
 interface EXPORTTOEXCELANNUAL("Export to
     Excel Holiday Report",
     DoExportToExcelAnnual)

 // ...

 function DoExportToExcelAnnual() {
  var objIoFile = new IO_FILE();
  objIoFile.SetFileName(Path_Make(Uuid(),
      "xls");
  objIoFile.AppendLn("<h3>Holiday Report
      %s</h3>".Format(Year(START_DATE)));
  var reportHtml =
      CalendarAux.GetYear(START_DATE);
  objIoFile.AppendLn(reportHtml);
  return ViewFile(objIoFile.FileName);
 }

 // ...
}
```

Listing 9 Pinset script for holiday report

```
pre {
 var months = "JAN FEB MAR APR MAY JUN JUL
     AUG SEP OCT NOV DEC".split(" ");
}

dataset HolidayReport over e : Employee {
 properties [name]
 grid {
  keys: months;
  header: key;
  body {
   var month = (months.indexOf(key) +
       1).asString().pad(2, "0", false);
   var dateSuffix = month + "/" + year;
   var absences = e.absences.select(ab |
       ab.day.endsWith(dateSuffix));
   var halfAbsences = absences.select(ab |
       ab.isHalf).size();
   var fullAbsences = absences.size() -
       halfAbsences;
   var fullDays = fullAbsences +
       0.5*halfAbsences;
   return fullDays == fullDays.round()
    ? fullDays.round()
    : fullDays;
  }
 }
}
```

further: while UGROUND duplicates structure in code and ROSE, Dandelion+ only keeps it in its linguistic model.

Overall, we conclude for RQ2 that Dandelion+ is able to improve UGROUND's industrial process in terms of rapid development, focused model management languages, and format flexibility.

8.3 Threats to validity

Internal validity. Internal validity is the degree of confidence that a causal relationship exists between the conducted experiment and the extracted conclusions.

In our evaluations, our conclusions are based on a comparison of solutions that we, the authors, implemented across various low-code platforms, including our own. Thus, especially for third-party tools, there is a risk that we have overlooked alternative solutions, potentially more optimal in certain aspects. To prevent this from affecting our conclusions, we studied all the tools used (OutSystems, Mendix, and BESSER) to the best of our abilities, and for the features under examination, we are confident that any features that we identified as missing in these tools are indeed absent, ensuring the validity of our conclusions at the time of this study.

Additionally, for RQ2, our understanding of UGROUND's process was validated by three employees who routinely follow this process, two of whom are authors of this paper.

External validity. External validity is the extent to which the findings of a study can be generalized.

For RQ1, we conducted our comparison on the basis of a single case study (Mudel), which may not fully represent the diverse range of low-code-based applications requiring code generation. To mitigate this limitation, we carefully designed our case study to include instances of all key features discussed in the paper, such as access control, ownership, users, roles, and behavior. In any case, to further validate our results, additional case studies could be conducted, including industrial ones. Likewise, our comparison covered only a subset of the existing low-code platforms. To minimize this threat, we selected two widely used professional low-code platforms for the comparison, as well as a recent academic tool built atop model-driven principles. We believe these three tools are highly representative of the broader landscape of low-code development platforms.

For RQ2, we compared Dandelion+'s process for building low-code solutions with that of UGROUND, since we

have detailed knowledge of this company's development process. However, we recognize that other software companies may follow slightly different processes. Therefore, our conclusions for RQ2 cannot be universally generalized for all companies.

Finally, we also acknowledge that the increased flexibility of Dandelion+ may entail a steeper learning curve, especially for individuals unfamiliar with model-driven development. Therefore, some training or documentation may be necessary for the usage of the tool.

9 Conclusions and future work

This paper has presented Dandelion+, a low-code platform for defining DSL-based platforms. The tool follows a model-driven foundation, enabling the management of both the structure and the behavior of such platforms. This paper has also introduced PLATFLOW, a workflow language catered for specifying behavior within low-code platforms. We have evaluated Dandelion+ against other academic and non-academic low-code platforms and its benefits within realistic low-code development processes.

As future work, we aim to enhance both the structural and behavioral dimensions of Dandelion+:

Structure-wise, we plan to port Dandelion's capabilities for defining graphical concrete syntaxes into Dandelion+. This will allow manipulating artifacts using a more suitable representation—e.g., when domains have a graph or states and transitions nature. We also intend to support the evolution of (meta-)models by integrating change-based persistence mechanisms [64] and model migration capabilities [44]. This will enable an incremental meta-model evolution while maintaining a trace of the changes and ensuring an efficient migration of existing models.

As per behavior, we would like to support defining state machines for classes, as they are a key behavioral component in UGROUND's models. This extension would allow infusing a state into objects, enabling their lifecycle management. In addition, we want to explore the feasibility of typing workflows encompassing a static analysis to detect structural errors at design time. Moreover, our template system currently computes all subpages when running an application. Instead, we want to only compute the needed subpages on demand, as in [65]. Finally, we aim to complement this template system with a drag-and-drop visual editor, similar to those found in other low-code platforms, so that users can design their interfaces without coding.

Finally, to mitigate the learning curve associated with Dandelion+'s flexibility, we aim to develop comprehensive documentation and tutorials. In particular, we aim to create a *guided tour for users new to model-driven development, potentially using frameworks like React Joyride,*[27] *to walk them through initial domain setup with integrated documentation links.* After that, we plan to carry out a user study to evaluate the usability of the tool.

A Constraints of the workflow meta-model

In this appendix, we include the OCL constraints of the workflow meta-model from Fig. 11.

First, the names of the input and output ports within a node must be different by name:

```
context Node
inv: inputPorts->isUnique(name)

context Node
inv: outputPorts->isUnique(name)
```

Moreover, input parameters of a workflow must also be unique:

```
context Workflow
inv: parameters->isUnique(name)
```

Second, workflow nodes admit an unlimited number of input ports by default (Node.inputPorts). However, some node types constrain their cardinality:

```
context InputNode
inv: inputPorts->isEmpty()

context EntityRetriever
inv: inputPorts->isEmpty()

context Text
inv: inputPorts->isEmpty()

context Evl
inv: inputPorts->size() = 1

context Pinset
inv: inputPorts->size() = 1

context Parser
inv: inputPorts->size() = 1

context Conditional
inv: inputPorts->size() = 1

context SensemakingStrategy
inv: inputPorts->size() = 1
```

Third, all node types have exactly one output port except Conditional, which has two of them—one per branch:

```
context Node
inv: not self.oclIsTypeOf(Conditional)
     implies outputPorts->size() = 1

context Conditional
inv: outputPorts->size() = 2
```

[27] https://react-joyride.com/.

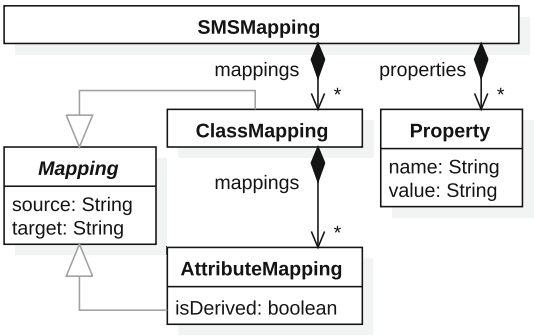


Fig. 40 Sensemaking strategies mapping meta-model

Specifically, both output ports of the Conditional node must be called "ok" and "ko" to correctly identify the execution branches:

```
context Conditional
inv: outputPorts->collect(name)->asSet()
     = Set{'ok', 'ko'}
```

Fourth, nodes that call other workflows cannot call the workflow they are part of (recursion is not allowed):

```
context Workflow
inv: nodes->forAll(n |
    n.oclIsKindOf(WfConsumer) implies
    n.references <> self
)
```

Finally, in WorkflowCalls, their input ports must match with the input parameters of the called workflow by name:

```
context WorkflowCall
inv:
let inputNames = references.parameters
    ->collect(name) in
let portNames = inputPorts
    ->collect(name) in
inputNames->asSet() = portNames->asSet()
```

B Sensemaking strategy mapping DSL

To define the mappings consumed by the SensemakingStrategy node, we employ a small, dedicated DSL. Figure 40 presents its meta-model.

Sensemaking strategy mappings relate meta-classes from the context meta-model to the target meta-model via ClassMappings, each containing AttributeMappings which may be derived. Non-derived mappings map attributes directly by name, whereas derived mappings use an EOL expression for the target where 'this' refers to the current instance. For instance:

- Non-derived: title ← name
- Derived: /x ← this.properties.size()

Additionally, mappings may define properties (e.g., the title of the chart or the labels of the axes) as pairs name/value. A deeper explanation of model sensemaking strategies and the rationale behind their mappings are available in [32].

Acknowledgements We want to thank UGROUND for funding this project. This work has also been partially supported by the Spanish MICINN (PID2021-122270OB-I00, TED2021-129381B-C21) and the EU Horizon 2020 Research and Innovation Programme under the Marie Sklodowska-Curie grant agreement No 813884.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

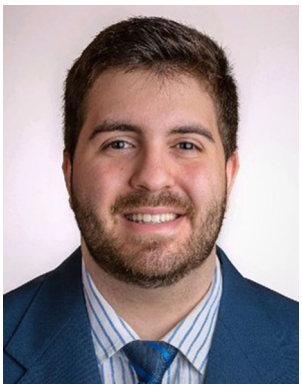
References

1. Alfonso, I., Conrardy, A., Sulejmani, A., et al.: Building BESSER: an open-source low-code platform. In: Proceedings of the 25th International Conference on Enterprise, Business-Process and Information Systems Modeling, Lecture Notes in Business Information Processing, vol. 511, pp. 203–212. Springer (2024)
2. Appian (last accessed in February 2025) https://appian.com
3. Atkinson, C.: Meta-modelling for distributed object environments. In: Proceedings of the 1st International Enterprise Distributed Object Computing Workshop, pp. 90–101 (1997)
4. Bancilhon, F., Spyratos, N.: Update semantics of relational views. ACM Trans. Database Syst. **6**(4), 557–575 (1981)
5. Brambilla, M., Cabot, J., Wimmer, M.: Model-Driven Software Engineering in Practice, 2nd edition. Synthesis Lectures on Software Engineering. Morgan & Claypool Publishers (2017)
6. Caspio (last accessed in February 2025) https://www.caspio.com
7. Cirit, Ç., Buzluca, F.: A UML profile for role-based access control. In: Proceedings of the 2nd International Conference on Security of Information and Networks, pp. 83–92. ACM (2009)
8. Corley, J., Syriani, E., Ergin, H.: Evaluating the cloud architecture of AToMPM. In: Proceedings of the 4rd International Conference on Model-Driven Engineering and Software Development, pp. 339–346. SciTePress (2016)
9. De La Vega, A., Sánchez, P., Kolovos, D.: Pinset: a DSL for extracting datasets from models for data mining-based quality analysis. In: Proceedings of the 11th International Conference on the Quality of Information and Communications Technology, pp. 83–91. IEEE (2018)
10. Di Ruscio, D., Kolovos, D.S., de Lara, J., et al.: Low-code development and model-driven engineering: two sides of the same coin? Softw. Syst. Model. **21**(2), 437–446 (2022)
11. Díez, A.: Recursive ontology-based systems engineering (2015). https://patents.google.com/patent/US9760345B2/en

12. de Lara, J., Vangheluwe, H.: AToM^3: a tool for multi-formalism and meta-modelling. In: Proceedings of the 5th International Conference on Fundamental Approaches to Software Engineering. Lecture Notes in Computer Science, vol. 2306, pp. 174–188. Springer (2002)
13. Elastic NV (last accessed in February 2025) Elasticsearch. https://www.elastic.co/elasticsearch
14. Facebook Open Source: React (2013). https://reactjs.org
15. Fowler, M.: Patterns of Enterprise Application Architecture. Addison-Wesley (2003)
16. Francis, M., Kolovos, D.S., Matragkas, N., et al: Adding spreadsheets to the MDE toolkit. In: Proceedings of the 16th International Conference on Model-Driven Engineering Languages and Systems, Lecture Notes in Computer Science, vol. 8107, pp. 35–51. Springer (2013)
17. Gamboa, M., Syriani, E.: Improving user productivity in modeling tools by explicitly modeling workflows. Softw. Syst. Model. **18**(4), 2441–2463 (2019)
18. Gamma, E., Helm, R., Johnson, R., et al.: Design Patterns: Elements of Reusable Object-Oriented Software, 1st edn. Addison-Wesley Professional (1994)
19. GEMOC Studio (last accessed in February 2025). https://gemoc.org/studio.html
20. Google AppSheet (last accessed in February 2025). https://cloud.google.com/appsheet
21. Indamutsa, A., Di Rocco, J., Di Ruscio, D., et al.: MDEForgeWL: towards cloud-based discovery and composition of model management services. In: Proceedings of the 24th International Conference on Model Driven Engineering Languages and Systems (Companion), pp. 118–127. IEEE (2021)
22. Indamutsa, A., Rocco, J.D., Almonte, L., et al.: Advanced discovery mechanisms in model repositories. Softw. Pract. Exp. **54**(11), 2214–2248 (2024)
23. Kahn, A.B.: Topological sorting of large networks. Commun. ACM **5**(11), 558–562 (1962)
24. Kelly, S., Tolvanen, J.: Domain-Specific Modeling-Enabling Full Code Generation. Wiley (2008)
25. Kissflow (last accessed in February 2025). https://kissflow.com
26. Kokaly, S.: Towards a structured workflow language for model management. In: Proceedings of Doctoral Symposium co-located with 17th International Conference on Model Driven Engineering Languages and Systems, CEUR Workshop Proceedings, vol. 1321. CEUR-WS.org (2014)
27. Kolovos, D., de la Vega, A.: Flexmi: a generic and modular textual syntax for domain-specific modelling. Softw. Syst. Model. **22**(4), 1197–1215 (2023)
28. Kolovos, D., Paige, R.F., Polack, F.: Eclipse development tools for Epsilon. In: Eclipse Summit Europe, p. 200, see also (2006a). https://eclipse.dev/epsilon/doc/
29. Kolovos, D., Paige, R.F., Polack, F.: The Epsilon Object Language (EOL). In: Proceedings of the 2nd European Conference on Model Driven Architecture-Foundations and Applications. Lecture Notes in Computer Science, vol. 4066, pp. 128–142. Springer (2006b)
30. Maróti, M., Kecskés, T., Kereskényi, R., et al.: Next generation (meta)modeling: web- and cloud-based collaborative tool infrastructure. In: Proceedings of the 8th Workshop on Multi-Paradigm Modeling, CEUR Workshop Proceedings, vol. 1237, pp. 41–60. CEUR-WS.org (2014)
31. Martínez-Lasaca, F., Díez, P., Guerra, E., et al.: Dandelion: a scalable, cloud-based graphical language workbench for industrial low-code development. J. Comput. Lang. **76**, 101217 (2023)
32. Martínez-Lasaca, F., Díez, P., Guerra, E., et al.: Model sensemaking strategies: exploiting meta-model patterns to understand large models. In: Proceedings of the 26th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, pp. 261–272. IEEE (2023b)
33. Martínez-Lasaca, F., Díez, P., Guerra, E., et al.: LowcoBot: towards chatting with low-code platforms. In: STAF 2024 Workshops, CEUR Workshop Proceedings, vol. 3727, pp. 66–76. CEUR-WS.org (2024)
34. Mendix (last accessed in February 2025). https://www.mendix.com
35. Microsoft PowerApps (last accessed in February 2025). https://powerapps.microsoft.com/en-us/low-code-platform
36. Mohd Kasim, N.N., Khalid, F.: Choosing the right learning management system (LMS) for the higher education institution context: a systematic review. Int. J. Emerg. Technol. Learn. **11**(6), 55–61 (2016)
37. OCL.: (2014). http://www.omg.org/spec/OCL
38. OMG.: Business process model and notation (2008a). https://www.bpmn.org
39. OMG.: Software and systems process engineering metamodel (2008b). https://www.omg.org/spec/SPEM/2.0/About-SPEM
40. OutSystems (last accessed in February 2025). https://www.outsystems.com
41. Paige, R.F., Kolovos, D.S., Rose, L.M., et al.: The design of a conceptual framework and technical infrastructure for model management language engineering. In: Proceedings of the 14th IEEE International Conference on Engineering of Complex Computer Systems, pp. 162–171. IEEE Computer Society (2009)
42. Rivera, J., Ruiz-González, D., López-Romero, F., et al.: Orchestrating atl model transformations. In: Proceedings of MtATL 2009 (2009)
43. Rose, L.M., Paige, R.F., Kolovos, D.S., et al.: The epsilon generation language. In: Proceedings of the 4th European Conference on Model Driven Architecture-Foundations and Applications. Lecture Notes in Computer Science, vol. 5095, pp. 1–16. Springer (2008)
44. Rose, L.M., Kolovos, D.S., Paige, R.F., et al.: Epsilon flock: a model migration language. Softw. Syst. Model. **13**(2), 735–755 (2014)
45. Rubio, A.D., de Lara, J.: Semantic digital twins for organizational development. In: Second International Workshop on Semantic Digital Twins (2021)
46. Russell, N., van der Aalst, W., ter Hofstede, A.: Workflow Patterns: The Definitive Guide. MIT Press, Cambridge (2016)
47. Sahay, A., Indamutsa, A., Di Ruscio, D., et al.: Supporting the understanding and comparison of low-code development platforms. In: Proceedings of the 46th Euromicro Conference on Software Engineering and Advanced Applications, pp. 171–178. IEEE (2020)
48. Salado-Cid, R., Vallecillo, A., Munir, K., et al.: SWEL: a domain-specific language for modeling data-intensive workflows. Bus. Inf. Syst. Eng. **66**(2), 137–160 (2024)
49. Salesforce Platform (last accessed in February 2025). https://www.salesforce.com/platform
50. Sánchez, B., Kolovos, D.S., Paige, R.: ModelFlow: towards reactive model management workflows. In: Proceedings of the 17th ACM SIGPLAN International Workshop on Domain-Specific Modeling, pp. 30–39. ACM (2019)
51. Sánchez, B., Kolovos, D., Paige, R.: To build, or not to build: model flow, a build solution for MDE projects. In: Proceedings of the 23rd International Conference on Model Driven Engineering Languages and Systems, pp. 1–11. ACM (2020)
52. Sánchez Cuadrado, J., Guerra, E., de Lara, J.: A component model for model transformations. IEEE Trans. Softw. Eng. **40**(11), 1042–1060 (2014)
53. Sirius (last accessed in February 2025). https://www.eclipse.org/sirius
54. Steinberg, D., Budinsky, F., Merks, E., et al.: EMF: Eclipse Modeling Framework, 2nd edn. Pearson Education (2008)
55. The Eclipse Foundation (2018) Eclipse Layout Kernel (ELK). https://www.eclipse.org/elk

56. The Eclipse Foundation (last accessed in February 2025a) EMF.cloud. https://www.eclipse.org/emfcloud
57. The Eclipse Foundation (last accessed in February 2025b) Modeling workflow engine. https://projects.eclipse.org/projects/modeling.emf.mwe
58. The Eclipse Foundation (last accessed in February 2025c) Sirius Web. https://www.eclipse.org/sirius/sirius-web.html
59. Tisi, M., Mottu, J., Kolovos, D.S., et al.: Lowcomote: Training the next generation of experts in scalable low-code engineering platforms. In: STAF Co-Located Events, CEUR Workshop Proceedings, vol. 2405, pp. 73–78. CEUR-WS.org (2019)
60. UGROUND (last accessed in February 2025). https://www.uground.com
61. van der Aalst, W., van Hee, K.M.: Workflow Management: Models, Methods, and Systems, Cooperative Information Systems. MIT Press, Cambridge (2002)
62. Wasowski, A., Berger, T.: Domain-Specific Languages-Effective Modeling, Automation, and Reuse. Springer (2023)
63. Xtext (last accessed in February 2025). http://www.eclipse.org/Xtext
64. Yohannis, A., Rodriguez, R.H., Polack, F., et al.: Towards efficient comparison of change-based models. J. Object Technol. **18**(2), 7:1-21 (2019)
65. Yohannis, A., Kolovos, D., García-Domínguez, A., et al.: Picto web: a tool for complex model exploration. In: Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, pp. 56–60. ACM (2022)
66. Zoho Creator (last accessed in February 2025). https://www.zoho.com/creator
67. Zweihoff. P., Naujokat, S., Steffen, B.: Pyro: generating domain-specific collaborative online modeling environments. In: Proceedings of the 22nd International Conference on Fundamental Approaches to Software Engineering. Lecture Notes in Computer Science, vol. 11424, pp. 101–115. Springer (2019)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Francisco Martínez-Lasaca is a Ph.D. candidate at the Computer Science Department of the Universidad Autónoma de Madrid. His research lies at the intersection of model-driven engineering, low-code platforms, and chatbots. He is also interested in both textual and graphical language engineering. Contact him at francisco.martinezl01@estudiante.uam.es, or visit https://fmlasaca.dev/.



Pablo Díez is CTO of UGROUND and head of the R&D Department. He studied Mathematics and Computer Science at C-SC College in Canton, MO, and holds an MA in Theoretical Mathematics from Central Michigan University. Pablo has a decade of experience in software development, starting his career in the research field of 3D face recognition and AI for student career prediction. At UGROUND, he has further developed the MDE and Semantic Engineering tools (mainly ROSE) to expand its nature to a full development and execution suite. Contact him at pdiez@uground.com, or visit https://www.uground.com/.



Esther Guerra is Full Professor at the Computer Science Department of the Universidad Autónoma de Madrid. Together with J. de Lara, she leads the modeling and software engineering research group (http://miso.es). She is interested in flexible modeling, metamodeling, domain-specific languages, and model transformation. Contact her at esther.guerra@uam.es, or visit http://www.ii.uam.es/ eguerra.



Juan de Lara is Full Professor at the Computer Science Department of the Universidad Autónoma de Madrid. Together with E. Guerra, he leads the modeling and software engineering research group (http://miso.es). His research interests are in model-driven engineering, automated software development, domain-specific languages, and chatbots. Contact him at juan.delara@uam.es, or visit http:// arantxa.ii.uam.es/ jlara/.